



Rapport de projet

Réalisé par

**Haitham El Abdioui,
Walid El Filali,
Youssef Kitabrhi**

Système de gestion de tickets **QuantumDesk**

sous la direction de :

Mme Kaouter KARBOUB Encadrant Académique

Remerciements

Nous tenons à exprimer notre profonde gratitude à notre professeure encadrante, **Madame Kaouter Karboub**, pour son soutien inestimable tout au long de ce projet en développement web et mobile. Son expertise et son engagement dans ce domaine ont grandement contribué à la réalisation de ce travail.

Madame Kaouter Karboub a su rendre les concepts parfois complexes du développement web et mobile accessibles et compréhensibles. Grâce à ses explications claires et à son approche pédagogique, nous avons pu maîtriser les principes de conception d'interfaces, d'architecture front-back et d'intégration d'API, ainsi que d'autres techniques essentielles comme la gestion d'état et le déploiement continu. Sa capacité à lier la théorie à des applications pratiques a été d'une grande aide pour orienter notre réflexion et structurer notre application.

Tout au long de nos échanges, **Madame Kaouter Karboub** a su nous orienter et nous encourager, même face aux difficultés rencontrées. Elle a non seulement partagé ses connaissances techniques, mais a également pris le temps de répondre à nos questions et de clarifier nos doutes, permettant ainsi de rendre ce projet plus robuste et mieux structuré. Ses conseils ont été essentiels pour comprendre la manière d'aborder la modélisation des composants, de formuler des hypothèses pertinentes et d'appliquer les meilleures pratiques de développement.

Nous remercions également **Madame Kaouter Karboub** pour sa patience et son implication dans le suivi régulier de nos progrès, en nous fournissant des retours constructifs à chaque étape de l'avancement du projet. Sa disponibilité et son soutien constant ont été des éléments déterminants dans la réussite de ce travail.

Nous adressons également nos plus sincères remerciements à nos familles pour leur amour, leur compréhension et leur soutien moral inconditionnel tout au long de ces mois de travail. Nous remercions Dieu pour la force et la persévérance qu'Il nous a accordées pour mener ce projet à bien. Nous tenons enfin à remercier chaleureusement notre établissement, **EMSI**, pour les ressources et l'environnement académique qu'il nous a offerts, sans lesquels ce travail n'aurait pu voir le jour.

En conclusion, nous exprimons notre reconnaissance à toutes les personnes qui, de près ou de loin, ont contribué à la réussite de ce projet. Leur soutien et leurs encouragements ont été précieux et ont grandement enrichi notre expérience académique et professionnelle.

Résumé

QuantumDesk est un système complet de gestion de tickets de support technique développé en ASP.NET Core 9.0 destiné à optimiser le traitement des demandes d'assistance au sein des organisations. Ce projet implémente une architecture en couches soigneusement structurée selon les patterns Repository et Service Layer, créant une séparation claire des responsabilités qui facilite la maintenance et l'évolution du système. La plateforme s'appuie sur Entity Framework Core pour la persistance des données dans SQL Server et adopte une approche conteneurisée via Docker pour simplifier le déploiement et la scalabilité. L'interface utilisateur, conçue avec HTML5, CSS3 et JavaScript, présente un thème distinctif "QuantumDesk" à la fois professionnel et intuitif, contribuant à une expérience utilisateur fluide sur tous les appareils grâce à son design responsive. Le système offre un ensemble riche de fonctionnalités métier, notamment la création et le suivi détaillé des tickets, un mécanisme d'assignation automatique basé sur des règles configurables, un système d'escalade intelligent pour les problèmes non résolus dans les délais impartis, une gestion hiérarchique des équipes de support, et un centre de notifications en temps réel. La solution inclut également une gestion sécurisée des pièces jointes, un historique complet des interventions pour une traçabilité totale, et des tableaux de bord analytiques pour mesurer la performance et identifier les tendances. L'automatisation intelligente constitue un point fort distinctif, permettant d'adapter finement les workflows aux processus spécifiques de chaque organisation. Le développement a relevé plusieurs défis techniques significatifs. La gestion des pièces jointes a été résolue par un système de stockage hybride associant métadonnées en base de données et fichiers physiques organisés selon une structure sécurisée. L'optimisation des performances face à un volume croissant de données a nécessité une stratégie d'indexation sophistiquée, des mécanismes de pagination efficaces et un chargement sélectif des relations. La complexité du moteur de règles pour l'assignation et l'escalade a été maîtrisée grâce à un système de priorité explicite permettant une configuration granulaire tout en évitant les conflits. QuantumDesk se distingue des solutions commerciales par sa flexibilité exceptionnelle, permettant une personnalisation complète sans les contraintes habituelles des systèmes propriétaires. Le coût total de possession est maîtrisé grâce à l'élimination des licences récurrentes, et l'intégration native avec l'écosystème existant est facilitée par une conception orientée vers l'interopérabilité. Les perspectives d'évolution sont nombreuses et prometteuses : intégration avec des plateformes de communication comme Teams et Slack, connexion aux systèmes de gestion de projet comme Jira et Trello, implémentation de fonctionnalités d'intelligence artificielle pour la catégorisation automatique et les recommandations de solutions, développement d'une API RESTful complète pour les applications mobiles et les intégrations tierces, enrichissement des capacités analytiques avec des visualisations interactives avancées, et internationalisation pour une adoption mondiale. Ce projet démontre comment une approche méthodique de conception, associée à des choix technologiques appropriés et à une attention constante portée aux besoins des utilisateurs, peut aboutir à un produit robuste et flexible qui répond efficacement aux défis quotidiens des équipes de support. QuantumDesk illustre le potentiel des solutions internes bien conçues à non seulement rivaliser avec les offres commer-

ciales, mais aussi à offrir une valeur ajoutée significative grâce à leur parfaite adéquation avec les processus spécifiques de l'organisation, contribuant ainsi à l'amélioration de l'expérience client et à l'optimisation des ressources.

Table des matières

Table des figures	vi
Liste des algorithmes	vi
1 Introduction	1
1.1 Contexte et problématique	1
1.2 Présentation de QuantumDesk comme solution	1
1.3 Objectifs du projet	2
1.4 Structure du rapport	3
2 Contexte et état de l’art	4
2.1 Besoin d’un système de gestion de tickets	4
2.1.1 Volume croissant des requêtes	4
2.1.2 Fonctionnalités attendues	4
2.2 Traçabilité	4
2.3 Priorisation	5
2.3.1 Critères de priorisation	5
2.3.2 Fonctionnalités clés	5
2.4 Coordination d’équipe	5
2.5 Analyse des performances	6
2.5.1 Indicateurs clés	6
2.5.2 Outils et livrables	6
2.6 Besoins émergents	6
2.7 Solutions existantes	7
2.7.1 Zendesk	7
2.7.2 JIRA Service Management	7
2.7.3 Freshdesk	8
2.7.4 ServiceNow	8
2.7.5 Autres solutions notables	8
2.7.6 Analyse comparative	9
2.8 Spécificités et positionnement de QuantumDesk	9
2.8.1 Positionnement distinctif	9
2.8.2 Fonctionnalités différenciatrices	10
3 Architecture et Stack Technologique	11
3.1 Vue d’ensemble de l’architecture	11
3.1.1 2.1.1 Flux de données et interactions entre les couches	12

3.1.2	Cycle de vie d'un ticket dans le système QuantumDesk	13
3.1.3	Assignation automatique d'un ticket	15
3.1.4	Escalade automatique d'un ticket	16
3.2	Stack technologique	18
3.2.1	Backend	18
3.2.2	Frontend	19
3.2.3	Outils et pratiques de développement	20
3.3	Diagramme de cas d'utilisation – Gestion des tickets	21
3.4	Modèle de données	22
3.4.1	Description détaillée des entités principales	23
3.4.2	Enums et Types de données	25
3.5	Patterns d'implémentation	26
3.5.1	Pattern Repository	26
3.5.2	Pattern Service Layer	27
3.5.3	Dependency Injection	28
3.5.4	Background Service Pattern	28
3.5.5	Observer Pattern	29
3.6	Sécurité et authentification	29
3.7	Authentification et Autorisation	29
3.7.1	ASP.NET Core Identity	29
3.7.2	Autorisation basée sur les rôles	30
3.8	Conception de l'Interface	32
3.8.1	Expérience utilisateur responsive	35
3.8.2	Vues spécialisées par rôle	36
3.9	Infrastructure DevOps et Déploiement Cloud	36
3.9.1	Conteneurisation avec Docker	36
3.9.2	Déploiement sur Azure App Service	37
3.9.3	Pipeline DevOps complet	37
3.9.4	Stratégie de montée en charge	38
4	Discussion et analyse critique	40
4.1	Points forts de l'implémentation	40
4.2	Défis rencontrés et solutions	42
4.3	Perspectives d'amélioration	43
	Conclusion Générale	45
	Bibliographie	46

Table des figures

3.1	Diagramme d'architecture en couches de l'application QuantumDesk	12
3.2	Diagramme de séquence illustrant le flux d'une requête de création de ticket à travers les différentes couches de l'application	13
3.3	Diagramme de séquence illustrant le cycle de vie complet d'un ticket à travers les différentes couches de l'application	15
3.4	Diagramme de séquence illustrant le processus d'assignation automatique d'un ticket dans le système QuantumDesk	16
3.5	Diagramme de séquence illustrant le processus d'escalade automatique d'un ticket dans le système QuantumDesk	17
3.6	Stack technologique backend utilisée pour l'application QuantumDesk	18
3.7	Capture d'écran du tableau de bord principal illustrant le thème QuantumDesk et ses composants visuels distinctifs	20
3.8	Diagramme cas d'utilisation complet de QuantumDesk	22
3.9	Diagramme entité-relation complet de QuantumDesk, montrant toutes les entités et leurs relations	26
3.10	Diagramme de Business Process Model and Notation de QuantumDesk	31
3.11	Interface Accueil – Synthèse des indicateurs clés	32
3.12	Interface Liste des Tickets – Recherche et filtres dynamiques	33
3.13	Interface Création de Ticket – Formulaire guidé	33
3.14	Interface Détail du Ticket – Vue modulaire et historique	34
3.15	Interface Gestion des Équipes – Attribution par double panneau	34
3.16	Volet Notifications et Historique – Suivi temps réel	35
3.17	Stack technologique globale utilisée pour l'application QuantumDesk	39
4.1	Architecture en couches de QuantumDesk	41
4.2	Interface de test des règles d'assignation montrant le résultat simulé	43
4.3	Documentation Swagger/OpenAPI pour l'API RESTful de QuantumDesk	44

Chapitre 1

Introduction

1.1 Contexte et problématique

Le support client et la gestion des incidents constituent des piliers fondamentaux dans la relation client pour toute entreprise moderne. Dans un environnement numérique en constante évolution, les organisations font face à des défis considérables :

- **Volume croissant de demandes** : la digitalisation des services a entraîné une augmentation exponentielle des demandes d'assistance.
- **Diversification des canaux de communication** : les clients s'attendent à pouvoir signaler des problèmes par email, téléphone, chat, réseaux sociaux ou portails web.
- **Complexité technique accrue** : les écosystèmes technologiques modernes impliquent des problèmes aux ramifications multiples nécessitant souvent l'intervention de plusieurs experts.
- **Attentes élevées des clients** : les standards de réactivité et de qualité de service se sont considérablement élevés.
- **Besoins de reporting précis** : les directions exigent des métriques détaillées pour évaluer la performance des équipes de support.

Sans système adapté, ces défis se traduisent par des conséquences directes : tickets perdus, délais de résolution allongés, communication défailante, frustration des clients et des agents de support, et impossibilité d'analyser objectivement la performance du service.

1.2 Présentation de QuantumDesk comme solution

QuantumDesk a été spécifiquement conçu pour relever ces défis en offrant une solution complète et intégrée de gestion des tickets. Plus qu'un simple outil de suivi, il s'agit d'une plateforme stratégique qui transforme la façon dont les organisations gèrent leurs interactions de support. Le système se distingue par plusieurs caractéristiques innovantes :

- **Omnicanal natif** : capacité à capturer et centraliser les demandes provenant de multiples sources.
- **Intelligence d'assignation** : algorithmes sophistiqués qui dirigent automatiquement les tickets vers les ressources les plus appropriées.
- **Workflows adaptables** : processus configurables qui s'adaptent aux méthodologies propres à chaque organisation.

- **Automatisation contextuelle** : déclenchement d'actions automatiques basées sur des règles métier précises.
- **Escalade proactive** : identification et remontée automatique des tickets à risque avant qu'ils ne deviennent critiques.
- **Interface intuitive** : expérience utilisateur optimisée pour maximiser la productivité des agents.
- **Analytique avancée** : tableaux de bord et rapports permettant d'identifier les tendances et opportunités d'amélioration.

1.3 Objectifs du projet

Le développement de QuantumDesk a été guidé par des objectifs stratégiques clairement définis :

- **Créer une plateforme centralisée pour la gestion des tickets de support** :
 - Éliminer les silos d'information entre départements.
 - Assurer une vue d'ensemble unifiée de toutes les interactions de support.
 - Simplifier l'accès à l'historique complet des échanges avec chaque client.
- **Améliorer l'efficacité du processus de résolution des problèmes** :
 - Réduire le temps moyen de résolution (MTTR) de 30% minimum.
 - Minimiser les transferts inutiles entre équipes.
 - Faciliter l'accès aux ressources nécessaires à la résolution.
 - Encourager la réutilisation des solutions pour des problèmes récurrents.
- **Offrir une traçabilité complète des actions prises sur chaque ticket** :
 - Documenter chaque interaction et modification.
 - Maintenir un journal d'audit inaltérable pour des raisons de conformité.
 - Permettre la reconstitution précise de la chronologie des événements.
 - Identifier les goulots d'étranglement dans les processus.
- **Fournir des mécanismes d'automatisation pour réduire les interventions manuelles** :
 - Automatiser les tâches répétitives à faible valeur ajoutée.
 - Standardiser les processus pour garantir leur cohérence.
 - Libérer du temps pour les agents afin qu'ils se concentrent sur les problèmes complexes.
 - Réduire les erreurs humaines dans le traitement des tickets.
- **Permettre une analyse détaillée des performances du service client** :
 - Mesurer précisément les KPIs critiques du support (temps de réponse, taux de résolution, satisfaction client).
 - Identifier les tendances et patterns dans les demandes de support.
 - Évaluer objectivement la performance individuelle et collective des agents.
 - Générer des insights actionnables pour l'amélioration continue des processus.

1.4 Structure du rapport

Ce rapport technique présente une analyse complète du projet QuantumDesk, articulée autour de trois axes principaux :

1. **Contexte et état de l'art** : examen des solutions existantes, forces et faiblesses, positionnement de QuantumDesk et tendances du marché.
2. **Architecture et stack technologique** : choix techniques, structure du système, modèles de données et technologies utilisées.
3. **Discussion et analyse critique** : évaluation des résultats par rapport aux objectifs, défis rencontrés, solutions apportées et perspectives d'évolution.

À travers ces trois chapitres, nous offrons une vision complète de QuantumDesk, depuis sa conception jusqu'à son implémentation et son potentiel d'évolution, illustrant comment cette solution répond aux défis contemporains de la gestion du support client.

Chapitre 2

Contexte et état de l'art

2.1 Besoin d'un système de gestion de tickets

Dans l'environnement professionnel contemporain, les systèmes de gestion de tickets sont devenus indispensables face à la complexification des infrastructures technologiques et à l'augmentation des attentes des utilisateurs. Ces systèmes répondent à plusieurs problématiques fondamentales auxquelles sont confrontées les organisations modernes :

2.1.1 Volume croissant des requêtes

Les départements IT et de support client font face à un nombre toujours plus important de demandes d'assistance, pour plusieurs raisons :

- *Numérisation accélérée* : la transformation digitale a multiplié les points de contact.
- *Expansion des services* : les entreprises proposent davantage de produits et services numériques.
- *Démocratisation de la technologie* : des utilisateurs aux niveaux de compétence variables sollicitent l'assistance.
- *Dépendance accrue* : la criticité des systèmes rend tout dysfonctionnement immédiatement problématique.

Une étude de HDI révèle que 67% des organisations ont connu une augmentation du volume de tickets de 15 à 30% au cours des trois dernières années, sans augmentation des effectifs de support.

2.1.2 Fonctionnalités attendues

Pour répondre à cette hausse, le système doit permettre :

- Le traitement parallèle d'un grand nombre de demandes.
- L'automatisation des tâches répétitives.
- La mise en place de workflows optimisés.
- L'équilibrage intelligent de la charge de travail.

2.2 Traçabilité

Le besoin de traçabilité répond à plusieurs impératifs :

- *Continuité du service* : reprise d'un ticket sans perte d'information.
- *Documentation* : constitution d'une base de connaissances.
- *Responsabilité* : identification des actions et des acteurs.
- *Conformité* : exigences réglementaires.
- *Résolution des litiges* : historique factuel.

En pratique, cela nécessite :

- Enregistrement horodaté de chaque interaction.
- Documentation des changements d'état et de priorité.
- Capture des métadonnées contextuelles.
- Archivage sécurisé des pièces jointes.
- Reconstitution chronologique de l'historique.

2.3 Priorisation

Dans un contexte où toutes les demandes ne peuvent être traitées simultanément, la priorisation devient cruciale :

2.3.1 Critères de priorisation

- *Impact business* : conséquences sur les opérations.
- *Criticité des systèmes* : systèmes essentiels.
- *SLAs* : respect des niveaux de service.
- *Ressources disponibles* : optimisation de l'allocation.
- *Interdépendances* : tickets bloquants.

2.3.2 Fonctionnalités clés

- Définition flexible des priorités (P1 à P4).
- Indicateurs visuels de priorité.
- Escalades automatiques et règles temporelles.
- Réévaluation dynamique.
- Vues filtrées pour les managers.

2.4 Coordination d'équipe

La résolution efficace des problèmes complexes nécessite :

- Collaboration transversale (réseau, sécurité, applicatif).
- Transfert contextuel avec informations complètes.
- Visibilité partagée de l'état global.
- Communication interne distincte.

- Escalade hiérarchique selon les besoins.

Fonctionnalités :

- Assignment à individus ou groupes.
- Notifications automatisées.
- Espaces de discussion internes.
- Partage de ressources et de connaissances.
- Workflows d'escalade.

2.5 Analyse des performances

Pour l'amélioration continue :

2.5.1 Indicateurs clés

- MTTR, temps de première réponse, taux de résolution.
- Identification des tendances et patterns.
- Évaluation de la performance individuelle.
- Mesure de la satisfaction client.
- Contrôle du respect des SLAs.

2.5.2 Outils et livrables

- Tableaux de bord personnalisables.
- Reporting avancé avec filtres.
- Alertes proactives.
- Intégration avec outils de sondage.
- Analyse prédictive des charges.

2.6 Besoins émergents

Nouveaux défis pour les organisations :

- Expérience omnicanal (email, chat, réseaux sociaux).
- Self-service via base de connaissances et chatbots.
- Intégration CRM, ERP, outils de supervision.
- IA pour catégorisation et suggestions automatiques.
- Conformité RGPD/CCPA.

Ces systèmes deviennent des plateformes stratégiques au cœur de la relation client et de l'efficacité opérationnelle.

2.7 Solutions existantes

Avant de développer *QuantumDesk*, une analyse approfondie du marché a été réalisée pour comprendre les forces et faiblesses des solutions existantes. Cette étude comparative a permis d'identifier les opportunités d'innovation et de différenciation pour notre solution.

2.7.1 Zendesk

Forces

- Richesse fonctionnelle : offre complète couvrant tous les aspects de la gestion des tickets.
- Interface utilisateur intuitive : design moderne et épuré, prise en main rapide.
- Intégrations nombreuses : plus de 1000 applications et connecteurs via sa marketplace.
- Omnicanalité : support natif e-mail, téléphone, chat, réseaux sociaux, messageries.
- Analytics avancés : tableaux de bord puissants et rapports personnalisables.
- Base de connaissances intégrée : solution complète de self-service.

Faiblesses

- Coût élevé : \$19–\$199 par agent et par mois, prohibitif à grande échelle.
- Complexité pour petites équipes : fonctionnalités parfois surdimensionnées.
- Personnalisation limitée : workflows et UI nécessitent des développements externes.
- Performance à grande échelle : ralentissements signalés pour gros volumes.
- Dépendance au fournisseur : peu de contrôle sur la roadmap.
- Flexibilité des rapports : personnalisation des métriques restreinte.

2.7.2 JIRA Service Management

Forces

- Adapté aux processus ITIL et DevOps.
- Intégration native avec Confluence, Bitbucket, etc.
- Workflows hautement personnalisables.
- Fonctionnalités projet héritées de JIRA.
- Moteur d'automatisation sophistiqué.
- Architecture scalable pour grands déploiements.

Faiblesses

- Courbe d'apprentissage abrupte ; interface dense.
- Fonctionnalités surdimensionnées pour un support basique.
- Administration complexe, nécessite compétences techniques.
- Portail client moins intuitif que des solutions spécialisées.
- Extensions payantes fréquentes.
- Peu adapté aux équipes non-techniques.

2.7.3 Freshdesk

Forces

- Simplicité d'utilisation, peu de formation requise.
- Bon rapport qualité-prix (à partir de \$15/agent/mois).
- Automatisation accessible aux non-techniques.
- Déploiement en quelques heures.
- Support multicanal unifié.
- Gamification pour motiver les agents.

Faiblesses

- Limité pour workflows complexes.
- Écosystème d'intégrations plus restreint.
- Reporting moins flexible.
- Fonctionnalités avancées en options payantes.
- Moins performant à très grande échelle.
- Personnalisation visuelle restreinte.

2.7.4 ServiceNow

Forces

- Couverture complète des processus ITIL et au-delà.
- Idéal pour les grandes entreprises multi-départements.
- Plateforme de développement très personnalisable.
- Gouvernance, conformité et audit avancés.
- Orchestration de processus inter-départements.
- Scalabilité mondiale.

Faiblesses

- Coût d'acquisition et de maintenance très élevé (>100 000 \$ /an).
- Implémentations longues (6–12 mois) et consultants spécialisés.
- Ressources techniques internes indispensables.
- Complexité excessive pour un support standard.
- Interface parfois jugée moins ergonomique.
- Fort vendor lock-in.

2.7.5 Autres solutions notables

HappyFox UI soignée, base de connaissances solide, tarification simple – mais moins d'intégrations.

Zoho Desk Excellent rapport qualité-prix, IA Zia intégrée – interface moins moderne.

ManageEngine ServiceDesk Plus On-premise, ITIL complet – UI datée, reporting limité.

OTRS & osTicket Open source, gratuit et ultra-personnalisable – nécessite ressources techniques et UI basique.

2.7.6 Analyse comparative

La matrice ci-dessous synthétise la position de chaque solution selon cinq critères clés :

Solution	Simplicité	Personnalisation	Coût	Fonctionnalités	Rapidité
Zendesk	★★★★☆	★★☆☆☆	★☆☆☆☆	★★★★★	★★★★★
JIRA SM	★★☆☆☆	★★★★★	★★★★☆	★★★★★	★★☆☆☆
Freshdesk	★★★★★	★★☆☆☆	★★★★☆	★★★★★	★★★★★
ServiceNow	★☆☆☆☆	★★★★★	★☆☆☆☆	★★★★★	★★☆☆☆

Problématiques communes

- Compromis simplicité vs personnalisation.
- Tarification prohibitive à large échelle.
- Frictions entre équipes techniques et non-techniques.
- Dépendance aux fournisseurs.
- Intégrations souvent spécifiques et coûteuses.
- Équilibre fonctionnalités / ergonomie mal calibré.

2.8 Spécificités et positionnement de QuantumDesk

Après analyse des solutions existantes et de leurs limitations, *QuantumDesk* a été développé comme une alternative stratégique pour répondre aux besoins spécifiques identifiés dans le projet GitHub.

2.8.1 Positionnement distinctif

- **Sur mesure**
 - Développée spécifiquement pour répondre aux besoins fonctionnels.
 - Centrée sur les workflows de support technique et l'expérience utilisateur.
 - Implémentation précise des fonctionnalités de gestion de tickets sans surcharge.
- **Évolutive**
 - Architecture modulaire basée sur les patterns Repository et Service Layer.
 - Séparation claire entre les couches pour faciliter les extensions.
 - Conception orientée interfaces pour remplacer ou ajouter des composants aisément.
- **Économique**
 - Solution auto-hébergée sans frais de licence par utilisateur.
 - Technologies standard (ASP.NET Core, SQL Server) minimisant les coûts d'infrastructure.
 - Code source accessible pour modifications internes sans dépendance externe.
- **Intégrée**
 - Prise en charge des intégrations avec les systèmes existants.
 - Authentification via ASP.NET Identity intégrable aux annuaires d'entreprise.
 - Schéma de données conçu pour importer/exporter facilement.

- **Personnalisable**
 - Workflows adaptables via règles d'assignation et d'escalade configurables.
 - Interface basée sur le design system « QuantumDesk-theme ».
 - Structure de permissions et rôles fine pour s'adapter à l'organisation.

2.8.2 Fonctionnalités différenciatrices

- **Système d'assignation automatique**
 - Classes `AssignmentRule` et `AssignmentRuleService`.
 - Règles configurables par catégorie, priorité, etc.
 - Interface d'administration dédiée à la gestion des règles.
- **Mécanisme d'escalade intelligent**
 - Services `EscalationRuleService` et `EscalationBackgroundService`.
 - Règles temporelles pour automatiquement escalader les tickets critiques.
 - Monitoring continu des SLA et alertes proactives.
- **Interface utilisateur moderne et responsive**
 - Design system « QuantumDesk-theme » en CSS.
 - Composants visuels distinctifs pour statuts, priorités, etc.
 - Mode sombre/clair avec transitions fluides et media queries.
- **Structure d'équipes flexible**
 - Modèles `SupportTeam` et `TeamMember`.
 - Assignation par équipe ou individu.
 - Tableaux de bord et métriques par équipe.
- **Adaptation mobile complète**
 - Navigation optimisée pour usage tactile.
 - Design responsive pour toutes tailles d'écran.
 - Optimisation des performances sur réseaux mobiles.
- **Base de connaissances intégrée**
 - Classe `KnowledgeBaseArticle` pour la gestion documentaire.
 - Liaison tickets–articles pour réutilisation de solutions.
 - Interface de recherche rapide des articles pertinents.

Chapitre 3

Architecture et Stack Technologique

3.1 Vue d'ensemble de l'architecture

QuantumDesk s'appuie sur une architecture en couches rigoureusement structurée, selon le modèle MVC (Model-View-Controller), enrichie par les patterns Repository et Service Layer. Cette organisation méthodique garantit une séparation optimale des préoccupations, facilitant ainsi la maintenance évolutive et la testabilité du système.

L'architecture se décompose en cinq couches distinctes, chacune avec des responsabilités clairement définies :

1. **Couche Présentation** : Composée des vues Razor et des contrôleurs ASP.NET Core MVC, cette couche constitue le point d'entrée de l'application. Elle gère l'interaction avec l'utilisateur, le rendu des interfaces et l'acheminement des requêtes. Les contrôleurs sont conçus pour être légers, déléguant le traitement métier aux services sous-jacents.
2. **Couche Service** : Véritable cœur de l'application, cette couche implémente toute la logique métier complexe et orchestre les opérations. Chaque service est spécialisé dans un domaine fonctionnel précis (tickets, utilisateurs, notifications, règles d'escalade, etc.) et encapsule les règles métier associées.
3. **Couche Repository** : Elle assure l'abstraction de l'accès aux données, isolant complètement la logique métier des détails d'implémentation liés à la persistance. Cette couche définit des interfaces génériques et spécifiques pour chaque entité, permettant un accès uniforme et flexible aux données.
4. **Couche Modèle** : Cette couche définit les entités du domaine et les structures de données utilisées dans l'application. Les modèles reflètent fidèlement les concepts métier du système de gestion de tickets, avec leurs propriétés, relations et contraintes.
5. **Couche Persistance** : Basée sur Entity Framework Core et SQL Server, cette couche gère l'interaction effective avec la base de données, le mapping objet-relationnel, et les opérations de lecture/écriture.

Une particularité notable de l'architecture est l'intégration de services d'arrière-plan (background services) pour les opérations automatisées, exploitant pleinement les capacités d'hébergement de services asynchrones de .NET Core. Ces services s'exécutent indépendamment des requêtes HTTP et assurent des tâches critiques comme l'escalade automatique des tickets en souffrance ou l'envoi de notifications.

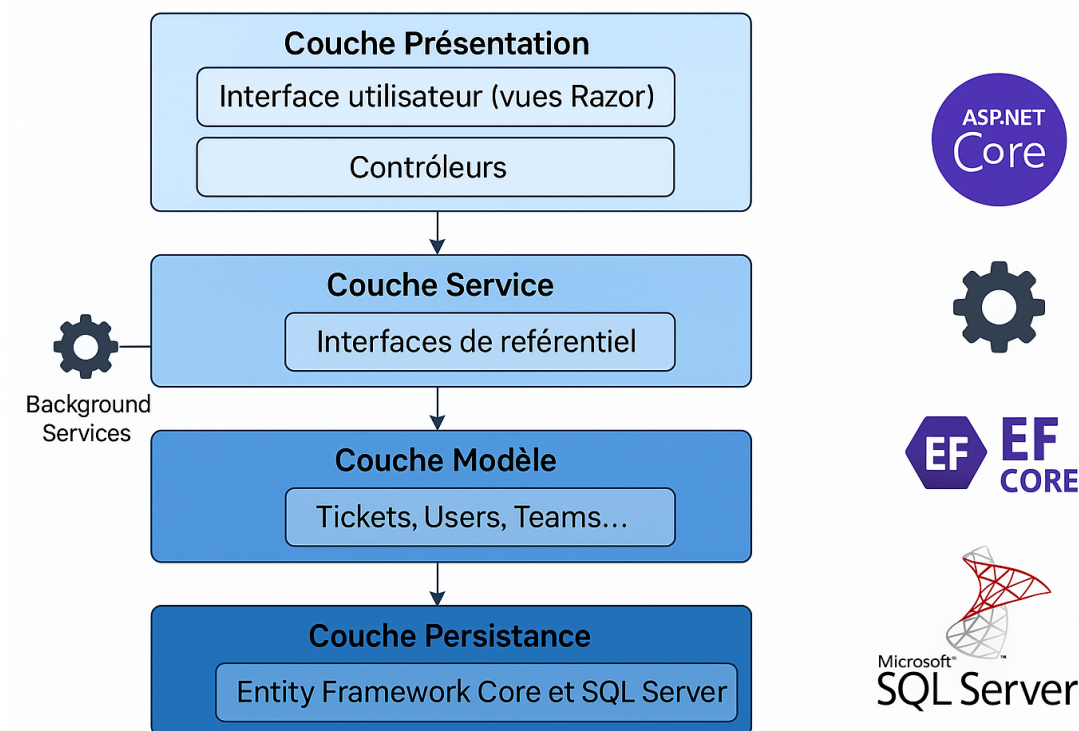


FIGURE 3.1 – Diagramme d’architecture en couches de l’application QuantumDesk

3.1.1 2.1.1 Flux de données et interactions entre les couches

Le parcours complet d’une requête dans l’architecture se déroule selon un circuit bien établi, garantissant une séparation claire des responsabilités :

1. **Initiation de la requête** : L’utilisateur interagit avec l’interface via son navigateur, déclenchant une requête HTTP vers l’application.
2. **Routage et contrôleur** : Le système de routage d’ASP.NET Core analyse l’URL et achemine la requête vers le contrôleur approprié. Par exemple, une demande de création de ticket est dirigée vers l’action `Create` du `TicketController`.
3. **Traitement par le contrôleur** : Le contrôleur valide les données d’entrée, puis délègue le traitement métier aux services correspondants, comme `ITicketService` pour la gestion des tickets.
4. **Exécution de la logique métier** : Le service sollicité exécute la logique métier requise, en appliquant les règles de gestion définies (droits, assignation automatique, notifications).
5. **Accès aux données via repositories** : Les services utilisent les repositories pour effectuer les opérations de données nécessaires, comme la persistance d’un nouveau ticket ou la récupération de l’historique.
6. **Opérations de base de données** : Les repositories traduisent les requêtes en opérations Entity Framework Core, exécutées sur la base SQL Server.
7. **Remontée des résultats** : Les données résultantes remontent des repositories aux services, puis aux contrôleurs.

8. **Génération de la vue** : Le contrôleur sélectionne la vue appropriée et lui transmet les données nécessaires via un modèle.
9. **Rendu de l'interface** : La vue Razor génère le HTML final qui est renvoyé au navigateur de l'utilisateur.

Cette séparation méthodique offre plusieurs avantages cruciaux pour un système comme **QuantumDesk** :

- **Isolement des changements** : une modification dans une couche n'affecte pas nécessairement les autres
- **Testabilité accrue** : chaque composant peut être testé indépendamment
- **Maintenabilité optimisée** : les développeurs peuvent se concentrer sur une couche spécifique
- **Évolutivité facilitée** : de nouvelles fonctionnalités peuvent être ajoutées en touchant minimalement le code existant

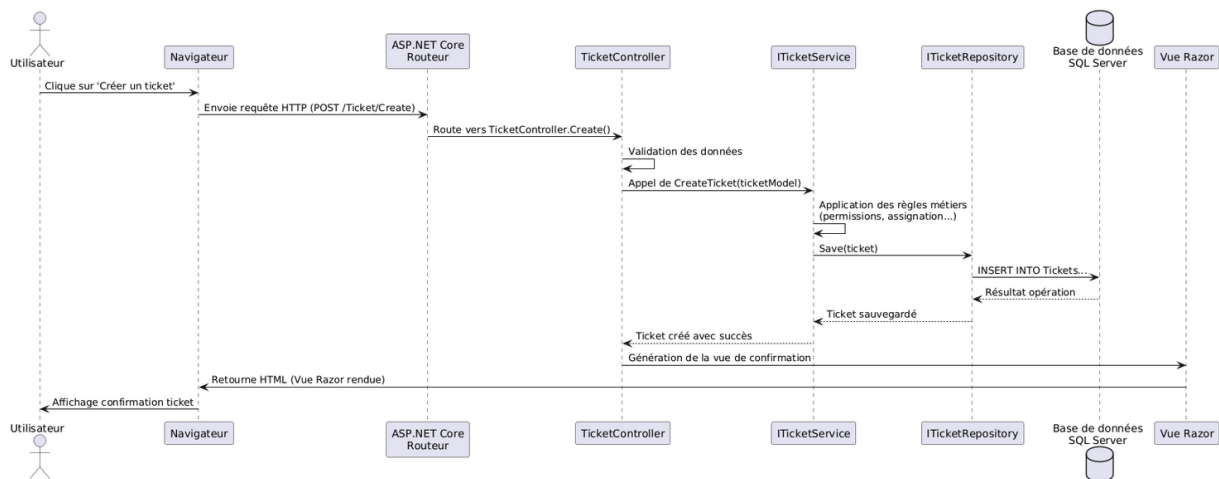


FIGURE 3.2 – Diagramme de séquence illustrant le flux d'une requête de création de ticket à travers les différentes couches de l'application

3.1.2 Cycle de vie d'un ticket dans le système QuantumDesk

Dans le cadre du système de gestion des tickets *QuantumDesk*, il est crucial de mettre en place un processus structuré et automatisé afin d'assurer le suivi efficace des demandes utilisateurs. Le cycle de vie d'un ticket décrit l'ensemble des étapes qu'un ticket parcourt depuis sa création jusqu'à sa clôture. Ce processus met en interaction plusieurs acteurs (utilisateur, agent support) et composants du système (contrôleur, services applicatifs, base de données). Le diagramme de séquence suivant illustre de manière détaillée ces différentes étapes.

1. Création du ticket

Lorsque l'utilisateur soumet une demande, le système initialise un nouveau ticket. Le contrôleur (*TicketController*) transmet les informations au service métier (*TicketService*) via la méthode `CreateTicketAsync`. Ce dernier effectue plusieurs opérations :

- Insertion du ticket dans la base de données.

- Application des règles d'assignation via le *AssignmentRuleService*.
- Envoi d'une notification de création via le *NotificationService*.

Une confirmation est ensuite retournée à l'utilisateur.

2. Traitement par l'agent

Une fois le ticket assigné, l'agent support peut entamer son traitement :

- **Prise en charge du ticket** : L'agent met à jour le statut du ticket en *InProgress* à l'aide de la méthode `UpdateStatusAsync`. Le système met à jour la base de données et envoie une notification de changement d'état.
- **Ajout d'un commentaire** : L'agent peut ajouter des précisions ou poser des questions via la méthode `AddCommentAsync`. Le commentaire est enregistré dans la table `TicketComments` et notifié aux parties concernées.

3. Résolution du ticket

Lorsque le problème est résolu, l'agent utilise la méthode `ResolveTicketAsync(ticketId)`. Le système procède alors aux actions suivantes :

- Mise à jour du statut du ticket en *Resolved*.
- Enregistrement de la date de résolution.
- Envoi d'une notification à l'utilisateur.

4. Fermeture du ticket

Enfin, l'utilisateur peut confirmer la résolution du ticket, ce qui entraîne sa fermeture définitive. Le système appelle la méthode `CloseTicketAsync` :

- Le statut est mis à jour en *Closed*.
- La date de fermeture est enregistrée.
- Une confirmation est retournée à l'utilisateur.

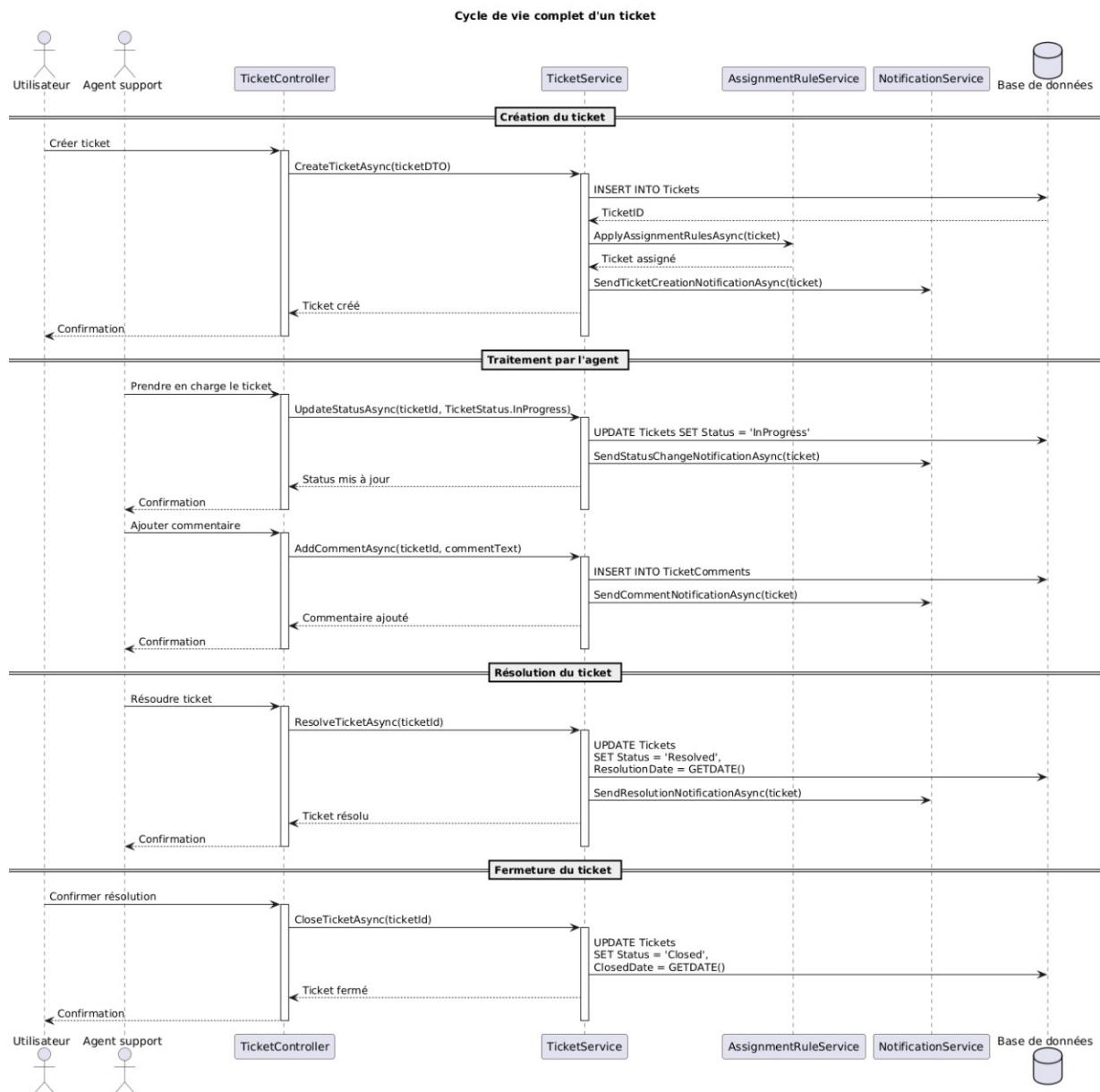


FIGURE 3.3 – Diagramme de séquence illustrant le cycle de vie complet d'un ticket à travers les différentes couches de l'application

3.1.3 Assignment automatique d'un ticket

Dans le système QuantumDesk, l'assignment automatique d'un ticket repose sur un ensemble de règles configurables qui déterminent à quel agent ou équipe un ticket doit être attribué après sa création. Le diagramme de séquence suivant décrit ce processus, qui vise à optimiser la distribution de la charge de travail parmi les agents disponibles.

- Le service *TicketService* appelle la méthode `ApplyAssignmentRulesAsync(ticket)` du *AssignmentRuleService*.
- Ce dernier interroge le *AssignmentRuleRepository* pour récupérer les règles d'assignment actives, triées par ordre de priorité.

- Chaque règle est évaluée par la méthode `MatchRule(ticket, rule)`.
- Si une règle correspond, la méthode `AssignTicketBasedOnRule(ticket, rule)` est invoquée.
- Lorsque la règle cible une équipe, le système appelle la méthode `GetLeastBusyTeamMemberAsync(teamId)` sur `SupportTeamService` pour identifier le membre avec le moins de tickets.
- L'ID de l'agent sélectionné est ensuite utilisé pour enregistrer l'assignation.
- Enfin, le service `TicketHistoryService` consigne ce changement dans l'historique en appelant `AddTicketHistoryAsync(ticketId, "AssignedTo", oldValue, newValue)`.

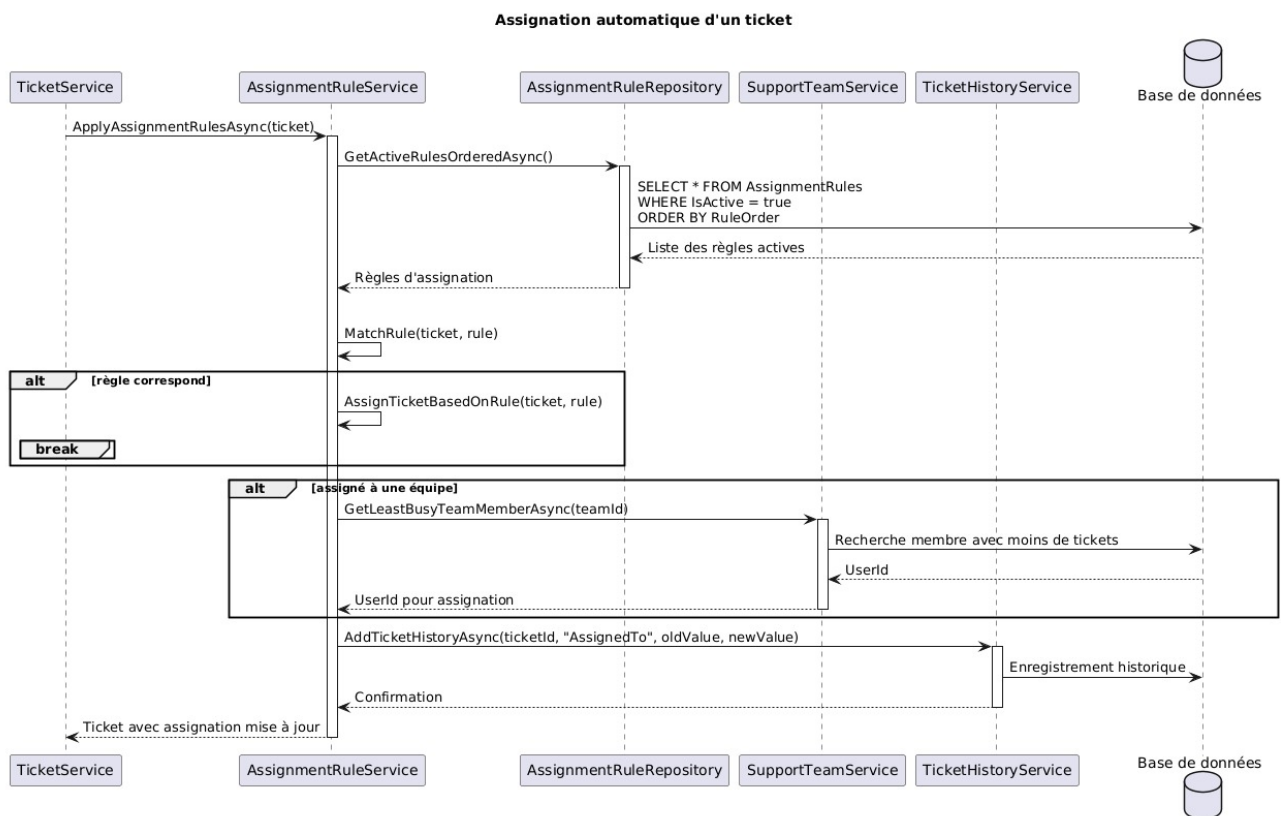


FIGURE 3.4 – Diagramme de séquence illustrant le processus d'assignation automatique d'un ticket dans le système QuantumDesk

3.1.4 Escalade automatique d'un ticket

Le système QuantumDesk intègre un mécanisme d'escalade automatique des tickets, permettant de garantir un traitement réactif en cas de dépassement du délai de résolution (SLA). Ce processus est exécuté de manière périodique par un service d'arrière-plan et repose sur un ensemble de règles définies en amont.

- Le `EscalationBackgroundService` exécute périodiquement la méthode `ExecuteAsync(CancellationToken)`.
- Il appelle ensuite `GetOpenTicketsWithExpiredSLAsAsync()` pour récupérer tous les tickets ouverts dont la date d'échéance est dépassée. Une requête SQL est exécutée pour cela.
- Pour chaque ticket en retard, le système recherche une règle d'escalade correspondante via `FindMatchingEscalationRuleAsync(ticket)`.

- Si une règle active correspond, l'escalade est appliquée via `EscalateTicketAsync(ticketId, ruleId)` :
 - Le ticket est marqué comme escaladé (`IsEscalated = true`),
 - Il est réassigné à un nouvel utilisateur ou une nouvelle équipe définie par la règle.
- Ensuite, des notifications sont envoyées aux destinataires définis par la règle à l'aide de `SendEscalationNotificationsAsync(ticket, rule)`. Cela génère des enregistrements dans la table `Notifications`.
- Le service attend un intervalle de temps défini (`Task.Delay`) avant de recommencer un nouveau cycle d'escalade.

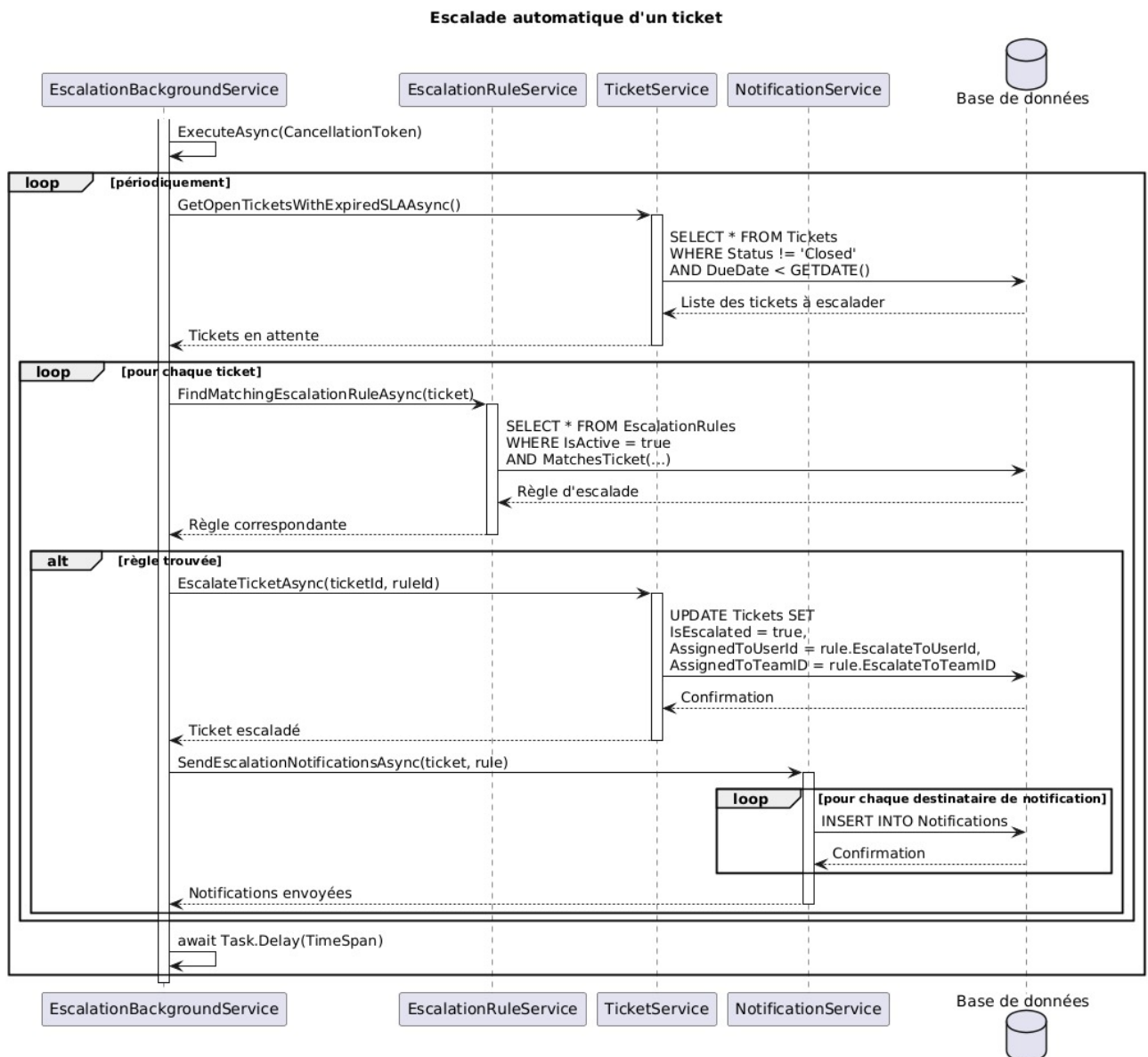


FIGURE 3.5 – Diagramme de séquence illustrant le processus d'escalade automatique d'un ticket dans le système QuantumDesk

3.2 Stack technologique

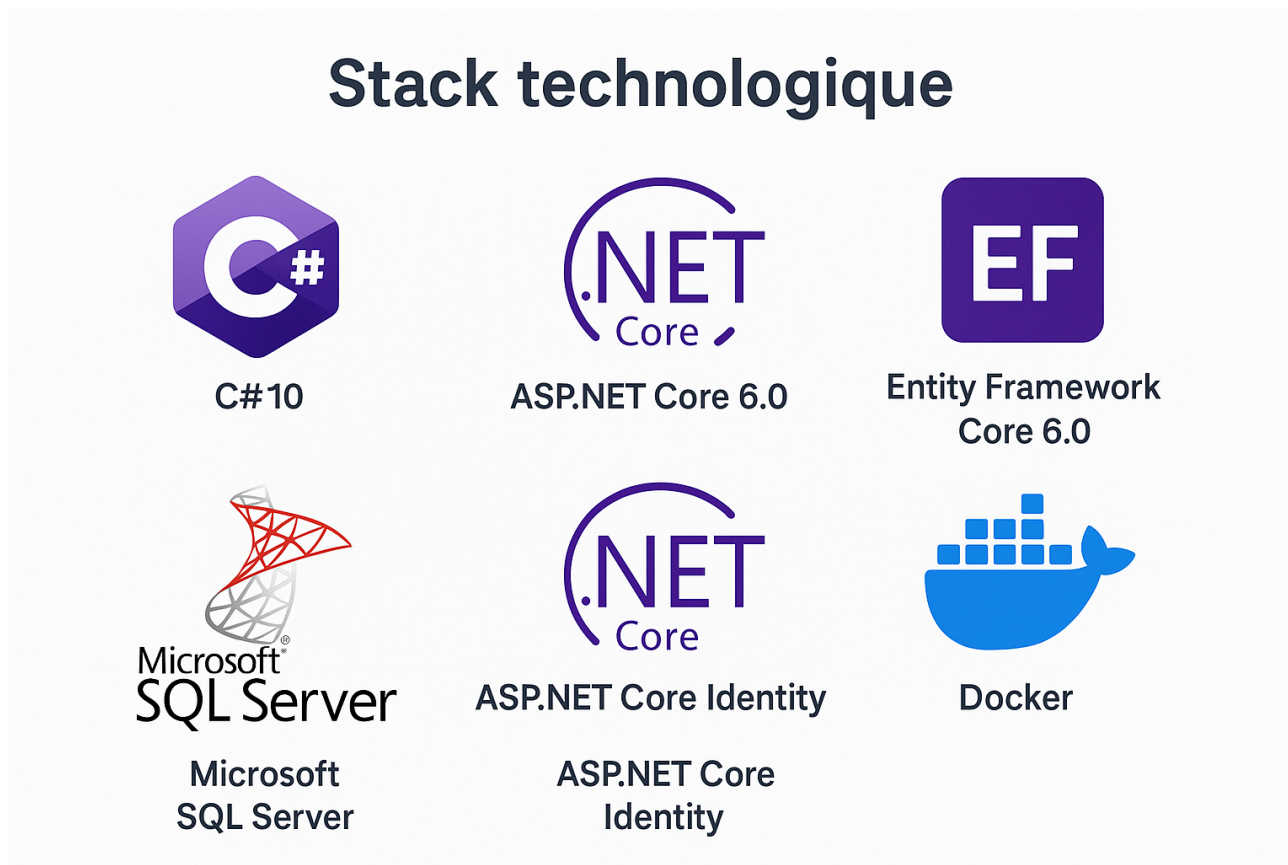


FIGURE 3.6 – Stack technologique backend utilisée pour l’application QuantumDesk

Le choix des technologies pour *QuantumDesk* a été guidé par des critères de performance, de maturité, de support à long terme et d’adéquation avec les besoins spécifiques d’un système de gestion de tickets moderne.

3.2.1 Backend

L’infrastructure backend repose sur des technologies Microsoft éprouvées et modernes :

- **Langage de programmation : C# 10**, sélectionné pour sa richesse syntaxique, sa sécurité de typage et ses avancées récentes (expressions `switch`, `records`, nullability, etc.). Les nouvelles fonctionnalités de C# 10 comme les *interpolated string handlers* et *file-scoped namespaces* ont permis d’écrire un code plus concis et maintenable.
- **Framework d’application : ASP.NET Core 9.0**, choisi pour sa performance exceptionnelle, sa conception modulaire et sa compatibilité *cross-platform*. Ce framework unifié permet d’intégrer harmonieusement les fonctionnalités MVC, Razor Pages et les API Web dans une même application.
- **ORM (Object-Relational Mapping) : Entity Framework Core 6.0**, qui fournit une abstraction robuste sur la base de données, avec des fonctionnalités avancées telles que :
 - Tracking des changements d’entités

- Migrations de schéma automatisées
- Requêtes LINQ typées et compilées
- Support des transactions distribuées
- Chargement différé (*lazy loading*) des relations
- **Base de données : Microsoft SQL Server**, sélectionnée pour sa robustesse, ses capacités de haute disponibilité et sa parfaite intégration avec l'écosystème .NET. Le système exploite des fonctionnalités avancées comme les *index columnstore* pour l'analytique et les requêtes temporelles pour le suivi historique.
- **Authentification et autorisation : ASP.NET Core Identity**, framework complet de gestion des identités, personnalisé pour les besoins spécifiques de QuantumDesk avec une extension du modèle utilisateur et une structure de rôles adaptée.
- **Conteneurisation : Support Docker** complet, avec des images *multi-stages* optimisées pour le développement et la production, facilitant le déploiement dans divers environnements (développement, test, production) et préparant le terrain pour une orchestration via Kubernetes si nécessaire.

Cette stack backend moderne offre un équilibre parfait entre performance, productivité de développement et facilité de maintenance, tout en garantissant une évolutivité pour les besoins futurs.

3.2.2 Frontend

L'expérience utilisateur s'appuie sur des technologies web modernes, avec une approche hybride combinant le rendu côté serveur et des améliorations JavaScript côté client :

- **Framework de vues : Razor Pages** avec ASP.NET Core MVC, permettant une structure claire des interfaces et un développement rapide. Cette approche offre un bon équilibre entre la simplicité du développement serveur et les interactions riches côté client.
- **Framework CSS : QuantumDesk-theme**, un framework personnalisé développé spécifiquement pour l'application. Construit sur les fondations de Bootstrap 5, il étend ses capacités avec :
 - Une palette de couleurs distinctive (bleu profond, turquoise, violet clair, rouge corail)
 - Des composants spécialisés pour l'affichage des tickets et leur statut
 - Des animations subtiles pour améliorer l'expérience utilisateur
 - Des styles optimisés pour les différents tableaux de bord et formulaires
- **JavaScript** : Une approche progressive utilisant principalement du JavaScript vanilla, enrichi ponctuellement avec jQuery pour des interactions complexes. Des modules spécifiques ont été développés pour :
 - La gestion dynamique des formulaires de tickets
 - L'affichage et le filtrage interactif des listes de tickets
 - Les indicateurs en temps réel et notifications
 - Les visualisations de données pour les tableaux de bord
- **Responsive Design** : Implémentation complète d'interfaces adaptatives, garantissant une expérience utilisateur optimale sur tout appareil, des écrans de bureau aux smartphones, avec des points d'arrêt (*breakpoints*) soigneusement définis et des interactions tactiles optimisées.

- **Iconographie** : Utilisation intensive de **Font Awesome 6**, avec une sélection d'icônes cohérente reflétant les concepts et actions du domaine (tickets, statuts, priorités, etc.).

Le framework CSS *QuantumDesk-theme* mérite une attention particulière. Il ne s'agit pas d'une simple personnalisation de Bootstrap, mais d'un système complet de design spécifiquement conçu pour les besoins d'une application de ticketing moderne. Il définit des variables CSS personnalisées, des composants spécialisés et des utilitaires adaptés au domaine.

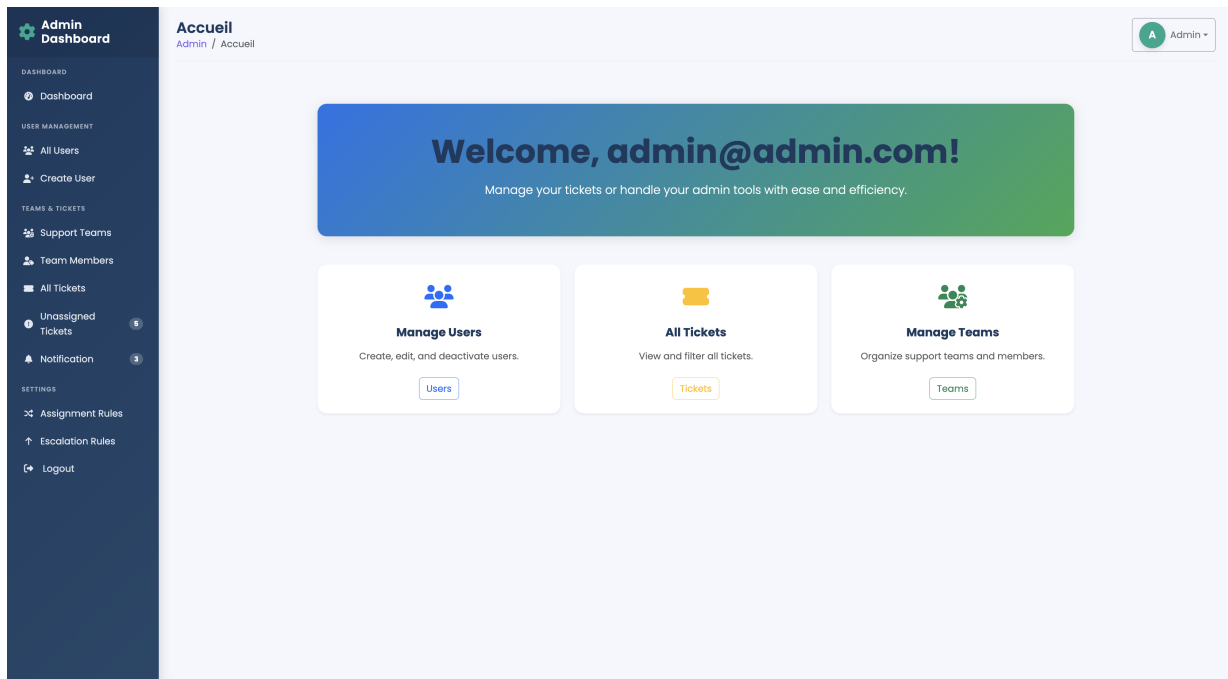


FIGURE 3.7 – Capture d'écran du tableau de bord principal illustrant le thème QuantumDesk et ses composants visuels distinctifs

3.2.3 Outils et pratiques de développement

Le développement de *QuantumDesk* s'est appuyé sur des pratiques d'ingénierie logicielle modernes et des outils favorisant la qualité et l'efficacité :

- **Contrôle de version** : **Git** avec une stratégie de branchement inspirée de GitFlow, comprenant :
 - Une branche **main** protégée pour le code de production
 - Une branche **develop** comme intégration continue des fonctionnalités
 - Des branches **feature/** pour le développement de nouvelles fonctionnalités
 - Des branches **hotfix/** pour les corrections urgentes
 - Des revues de code systématiques via *pull requests*
- **CI/CD (Intégration Continue / Déploiement Continu)** : Pipeline configuré pour **Azure DevOps** avec :
 - Builds automatisés à chaque commit
 - Exécution des tests unitaires et d'intégration

- Analyse statique du code via **SonarQube**
- Déploiement automatisé dans l'environnement de test après validation
- Déploiement en production via approbation manuelle

Ces pratiques ont permis de maintenir un haut niveau de qualité tout au long du développement, en détectant les problèmes potentiels au plus tôt et en facilitant la collaboration entre les membres de l'équipe.

3.3 Diagramme de cas d'utilisation – Gestion des tickets

Le diagramme de cas d'utilisation ci-dessous présente les différentes interactions possibles entre les acteurs du système QuantumDesk et les fonctionnalités liées à la gestion des tickets. Il met en évidence les responsabilités respectives des utilisateurs standards, des agents de support, des administrateurs ainsi que du système d'automatisation.

- Le **Système d'Automatisation** déclenche automatiquement certaines actions critiques comme l'assignation des tickets ou leur escalade en cas de dépassement des délais de traitement.
- L'**Utilisateur Standard** dispose de fonctions essentielles telles que la création d'un ticket, l'ajout de commentaires, la consultation de ses tickets et la clôture lorsqu'un problème est résolu.
- L'**Agent de Support** bénéficie de fonctionnalités avancées : il peut consulter les détails complets d'un ticket (incluant les historiques, les pièces jointes, et l'état actuel), assigner un ticket à un collègue ou à une équipe, et ajouter des commentaires internes non visibles par l'utilisateur.
- L'**Administrateur**, en plus d'avoir accès aux actions d'un agent, peut également filtrer, trier et modifier la priorité ou la catégorie d'un ticket.

Des relations d'extension (« *extends* ») permettent de modéliser des fonctionnalités optionnelles ou conditionnelles comme : réouvrir un ticket fermé, visualiser les modifications, ou assigner manuellement un ticket. Ces extensions enrichissent la flexibilité du système en adaptant les actions aux rôles et au contexte.

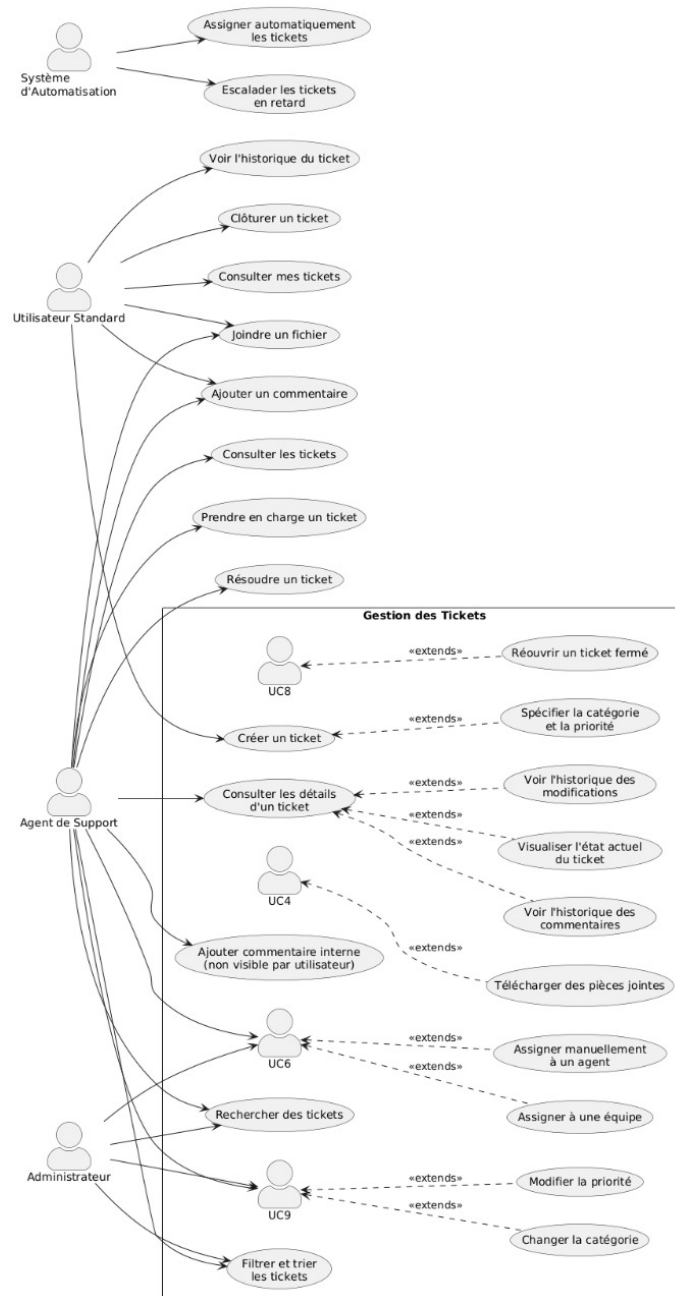


FIGURE 3.8 – Diagramme cas d'utilisation complet de QuantumDesk

3.4 Modèle de données

Le modèle de données de QuantumDesk a été conçu pour capturer efficacement tous les aspects d'un système moderne de gestion de tickets, avec une attention particulière à la traçabilité, la flexibilité et les relations entre entités.

Au centre de ce modèle se trouve l'entité **Ticket**, autour de laquelle gravitent plusieurs entités complémentaires qui enrichissent le système :

- **Ticket** : L'entité principale représentant une demande d'assistance, avec ses attributs

d'identification, de catégorisation, de priorisation et de suivi temporel. Elle maintient des relations avec les entités utilisateur (créateur et assigné), les équipes de support, ainsi que l'historique des modifications et interactions.

- **User** : Extension personnalisée d'ASP.NET Identity, cette entité représente tous les utilisateurs du système, qu'ils soient clients, agents de support ou administrateurs. Elle comprend des données professionnelles (nom, prénom) et des informations de suivi (date de création, dernière connexion).
- **TicketComment** : Enregistre tous les échanges liés à un ticket, avec distinction entre commentaires publics (visibles par tous) et notes internes (réservées au personnel de support). Cette séparation permet une communication transparente avec les utilisateurs tout en maintenant des discussions techniques internes.
- **Attachment** : Gère les pièces jointes associées aux tickets, avec stockage sécurisé des fichiers et métadonnées associées (nom, type, taille, auteur de l'upload).
- **TicketHistory** : Consigne minutieusement chaque modification apportée à un ticket, créant un journal d'audit complet avec l'utilisateur responsable, la date, le champ modifié et les valeurs avant/après changement. Cette traçabilité est cruciale pour le suivi de la qualité et la résolution des litiges.
- **SupportTeam** : Organise les agents de support en équipes fonctionnelles ou thématiques, permettant une assignation collective des tickets et une gestion des responsabilités par domaine d'expertise.
- **TeamMember** : Table de jointure établissant la relation many-to-many entre utilisateurs et équipes, avec des métadonnées complémentaires comme la date d'intégration.
- **AssignmentRule** : Définit les règles d'attribution automatique des tickets basées sur divers critères (catégorie, priorité), permettant l'acheminement intelligent des demandes vers les agents ou équipes appropriés.
- **EscalationRule** : Spécifie les conditions d'escalade automatique des tickets (délais de résolution dépassés, priorité critique), assurant que les problèmes importants reçoivent l'attention nécessaire.
- **KnowledgeBaseArticle** : Base de connaissances permettant de documenter les solutions aux problèmes courants, participant à la capitalisation des connaissances et à l'auto-résolution par les utilisateurs.
- **Notification** : Système d'alertes internes informant les utilisateurs des événements importants, comme l'assignation d'un ticket ou un dépassement de délai.

3.4.1 Description détaillée des entités principales

Ticket L'entité **Ticket** constitue le cœur du système QuantumDesk, conçue pour suivre minutieusement chaque demande d'assistance de sa création à sa résolution :

- **Identifiants et description** :
 - TicketID : Identifiant unique auto-incrémenté
 - Title : Titre concis décrivant le problème (255 caractères max)
 - Description : Description détaillée du problème (texte long)
- **Catégorisation et priorisation** :

- Category : Énumération (SoftwareIssue, HardwareIssue, NetworkProblem, GeneralInquiry)
- Priority : Énumération (Low, Medium, High, Critical)
- Status : Énumération (Open, InProgress, Resolved, Closed)
- **Attribution et responsabilité :**
 - CreatedByUserId : ID de l'utilisateur ayant créé le ticket
 - AssignedToUserId : ID de l'agent assigné (nullable)
 - AssignedToTeamID : ID de l'équipe assignée (nullable)
- **Suivi temporel :**
 - CreatedDate : Date/heure de création
 - UpdatedDate : Date/heure de dernière modification
 - DueDate : Date d'échéance prévue (nullable)
 - ResolutionDate : Date/heure de résolution (nullable)
 - ClosedDate : Date/heure de clôture (nullable)
- **Métadonnées :**
 - Source : Origine du ticket (Web, Email, Téléphone, etc.)
 - IsEscalated : Indicateur si le ticket a été escaladé
- **Relations (propriétés de navigation) :**
 - CreatedByUser : Utilisateur ayant créé le ticket
 - AssignedToUser : Agent assigné au ticket
 - AssignedToTeam : Équipe assignée au ticket
 - TicketHistories : Collection d'entrées d'historique
 - TicketComments : Collection de commentaires
 - TicketAttachments : Collection de pièces jointes

Cette modélisation complète permet un suivi précis de chaque aspect du cycle de vie d'un ticket, facilitant la génération de rapports détaillés et l'application de règles métier complexes.

User L'entité **User** étend le modèle standard d'ASP.NET Core Identity pour l'adapter aux besoins spécifiques de QuantumDesk :

- **Informations d'identification :**
 - Propriétés standard héritées d'IdentityUser (Id, UserName, Email, PasswordHash, etc.)
 - FirstName et LastName : Nom et prénom de l'utilisateur
- **Suivi d'activité :**
 - CreatedDate : Date de création du compte
 - LastLoginDate : Date/heure de dernière connexion
 - IsActive : État actif ou inactif du compte
- **Relations multiples :**
 - CreatedTickets : Tickets créés par l'utilisateur
 - AssignedTickets : Tickets assignés à l'utilisateur (agent)

- ManagedTeams : Équipes gérées par l'utilisateur
- TeamMemberships : Équipes dont l'utilisateur est membre
- Comments : Commentaires postés par l'utilisateur
- Attachments : Pièces jointes téléversées
- TicketChanges : Modifications de tickets effectuées
- Articles : Articles de knowledge base rédigés
- Notifications : Notifications reçues

Cette conception riche permet de suivre précisément les activités et responsabilités de chaque utilisateur dans le système, qu'il soit un client rapportant des problèmes ou un agent de support les résolvant.

3.4.2 Enums et Types de données

Pour garantir la cohérence des données et faciliter l'interface utilisateur, QuantumDesk utilise plusieurs types énumérés qui formalisent les valeurs possibles pour certains aspects clés du système :

- **TicketStatus** :
 - Open : Ticket nouvellement créé, pas encore traité
 - InProgress : Ticket en cours de traitement par un agent
 - Resolved : Problème résolu, en attente de confirmation
 - Closed : Ticket définitivement fermé
- **TicketPriority** :
 - Low : Impact mineur, résolution non urgente
 - Medium : Impact modéré, résolution dans les délais standard
 - High : Impact significatif, résolution prioritaire
 - Critical : Impact majeur, résolution immédiate requise
- **TicketCategory** :
 - SoftwareIssue : Problèmes liés aux logiciels
 - HardwareIssue : Problèmes liés au matériel
 - NetworkProblem : Problèmes de connectivité réseau
 - GeneralInquiry : Questions générales et demandes d'information

Ces énumérations sont utilisées non seulement pour typer fortement les propriétés correspondantes dans les entités, mais aussi pour construire des interfaces utilisateur adaptatives (code couleur par priorité, workflows différenciés par statut) et pour implémenter la logique métier (règles d'assignation basées sur la catégorie, SLAs différenciés selon la priorité).

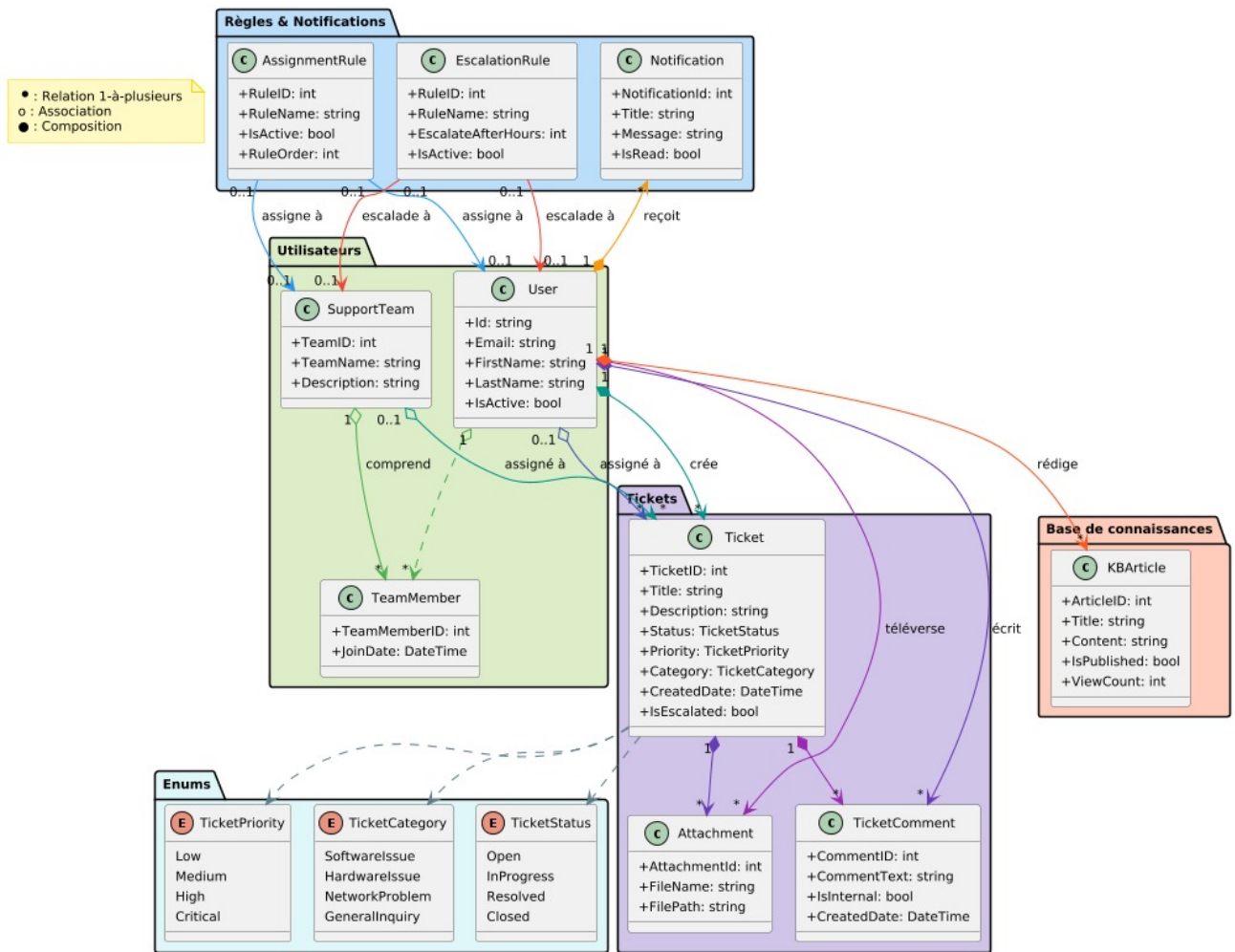


FIGURE 3.9 – Diagramme entité-relation complet de QuantumDesk, montrant toutes les entités et leurs relations

3.5 Patterns d'implémentation

L'architecture de QuantumDesk s'appuie sur plusieurs patterns de conception éprouvés, soigneusement sélectionnés et adaptés pour répondre aux besoins spécifiques d'un système de gestion de tickets moderne.

3.5.1 Pattern Repository

Le pattern Repository a été implémenté pour créer une abstraction complète entre la logique métier et l'accès aux données, offrant plusieurs avantages cruciaux :

- **Abstraction des sources de données** : Les services n'interagissent qu'avec des interfaces de repository, les isolant totalement des détails d'implémentation liés à la persistance (SQL Server via Entity Framework Core).
- **Structure à deux niveaux** :

- Interface générique : `IRepository<T>` définit les opérations CRUD communes à toutes les entités (`GetAllAsync`, `GetByIdAsync`, `AddAsync`, `UpdateAsync`, `DeleteAsync`).
- Interfaces spécialisées : Par exemple, `ITicketRepository` étend l'interface générique avec des méthodes spécifiques aux tickets (`GetTicketsByUserIdAsync`, `GetTicketsByStatusA` etc.).
- **Implémentations concrètes :**
 - Classe de base : `Repository<T>` implémente `IRepository<T>` en utilisant Entity Framework Core.
 - Classes spécialisées : Par exemple, `TicketRepository` implémente `ITicketRepository` avec la logique spécifique aux tickets, incluant les jointures nécessaires et les optimisations de requêtes.
- **Bénéfices tangibles :**
 - Testabilité accrue
 - Flexibilité (changement de moteur de base de données sans impacter les services)
 - Optimisation ciblée
 - Réutilisabilité du code commun

L'implémentation des repositories dans QuantumDesk inclut également des fonctionnalités avancées comme le chargement *eager* des relations pertinentes, la pagination des résultats pour les listes volumineuses, et l'utilisation de requêtes LINQ optimisées.

3.5.2 Pattern Service Layer

La couche service constitue le cœur fonctionnel de QuantumDesk, encapsulant toute la logique métier et orchestrant les opérations complexes :

- **Organisation par domaine fonctionnel :**
 - `TicketService` : Gestion du cycle de vie des tickets
 - `UserService` : Gestion des utilisateurs et profils
 - `NotificationService` : Système de notifications
 - `AssignmentRuleService` : Application des règles d'assignation
 - `EscalationRuleService` : Gestion de l'escalade des tickets
 - etc.
- **Structure cohérente :**
 - Interfaces (ex : `ITicketService`)
 - Implémentations concrètes avec encapsulation des règles métier
- **Responsabilités :**
 - Validation des données d'entrée
 - Application des règles métier complexes
 - Coordination entre plusieurs repositories
 - Gestion des transactions
 - Émission d'événements et notifications

Par exemple, la méthode `CreateTicketAsync` du `TicketService` applique :

- Validation approfondie des données
- Attribution de valeurs par défaut (statut, dates)
- Assignment automatique selon les règles configurées
- Création d'une entrée d'historique pour l'audit
- Notification aux personnes concernées

Cette couche service assure que toutes les règles métier sont centralisées et appliquées de manière cohérente, indépendamment du point d'entrée (interface web, API, intégration future avec d'autres systèmes).

3.5.3 Dependency Injection

L'architecture de QuantumDesk exploite pleinement le pattern *Dependency Injection*, nativement supporté par ASP.NET Core, pour garantir un couplage faible entre les composants :

- **Configuration centralisée** : Toutes les dépendances sont déclarées dans le fichier `Program.cs`, offrant une vue d'ensemble claire des services disponibles.
- **Durée de vie des services** :
 - **Scoped** : La plupart des services sont liés au cycle de vie HTTP
 - **Singleton** : Services statiques ou sans état (config, cache)
 - **Transient** : Créés à chaque injection
- **Injection par constructeur** : Toutes les dépendances sont injectées via les constructeurs, rendant explicites les besoins et facilitant les tests unitaires.

Cette approche offre plusieurs avantages :

- Couplage faible
- Testabilité accrue
- Configuration centralisée
- Intégration fluide avec les patterns Repository et Service Layer

3.5.4 Background Service Pattern

QuantumDesk implémente le pattern *Background Service* via le support natif d'ASP.NET Core pour les services hébergés, permettant l'exécution de tâches critiques en arrière-plan, indépendamment du cycle de vie des requêtes HTTP :

- **EscalationBackgroundService** : Vérifie périodiquement les tickets en attente depuis trop longtemps et applique les règles d'escalade configurées.
- **NotificationCleanupService** : Supprime régulièrement les anciennes notifications pour optimiser la base de données.
- **Caractéristiques d'implémentation** :
 - Exécution asynchrone via `ExecuteAsync` et le pattern Producer/Consumer
 - Support de l'annulation via `CancellationToken`
 - Gestion des exceptions avec politiques de retry
 - Logging détaillé
 - Intervalles configurables

Cette architecture permet d'assurer des opérations critiques même en l'absence d'interaction utilisateur, améliorant ainsi la réactivité du système et la qualité du service.

3.5.5 Observer Pattern

Le pattern *Observer* est mis en œuvre dans QuantumDesk à travers le système de notification, permettant de découpler les actions déclenchantes (changements d'état des tickets) des réactions correspondantes (notifications aux utilisateurs) :

- **Implémentation via la couche service** : Au lieu d'une implémentation classique avec des interfaces *Observer*, le pattern est réalisé via des appels de service, en profitant de l'injection de dépendances :
 - Les services qui génèrent des événements (comme *TicketService*) dépendent de *INotificationService*
 - Quand un événement significatif se produit, le service pertinent appelle *INotificationService* pour créer des notifications
- **Événements surveillés** :
 - Création de ticket
 - Changement de statut
 - Assignment à un agent ou une équipe
 - Ajout de commentaire
 - Escalade de ticket
 - Approche de la date d'échéance
- **Notification ciblée** : Les notifications sont créées spécifiquement pour les utilisateurs concernés par l'événement, en fonction de leur rôle et relation avec le ticket (créateur, assigné, membre d'équipe).
- **Persistence et affichage** : Les notifications sont persistées en base de données et affichées dans l'interface utilisateur via un composant dédié, avec indicateur visuel pour les éléments non lus.

Cette implémentation du pattern *Observer* permet de maintenir tous les utilisateurs informés des événements pertinents, tout en conservant une architecture découplée et facile à étendre.

3.6 Sécurité et authentification

La sécurité est une préoccupation majeure dans QuantumDesk, étant donné la nature potentiellement sensible des informations traitées dans un système de tickets. Une approche multi-couches a été adoptée pour garantir la confidentialité, l'intégrité et la disponibilité des données.

3.7 Authentification et Autorisation

3.7.1 ASP.NET Core Identity

Le système d'authentification repose sur *ASP.NET Core Identity*, framework mature et sécurisé pour la gestion des identités, avec plusieurs personnalisations pour répondre aux besoins spécifiques de QuantumDesk :

- **Extension du modèle utilisateur** : Le modèle `User` étend la classe `IdentityUser` standard avec des propriétés métier spécifiques (nom, prénom, statut d'activité) et des relations avec d'autres entités (tickets, équipes, commentaires, articles, notifications).
- **Configuration adaptée** :
 - Politique de mot de passe équilibrée, privilégiant la longueur sur la complexité excessive
 - Session de connexion prolongée avec option "Se souvenir de moi"
 - Verrouillage temporaire du compte après tentatives infructueuses
 - Support optionnel de l'authentification à deux facteurs pour les comptes sensibles
- **Stockage sécurisé** :
 - Mots de passe hachés et salés avec algorithmes modernes (PBKDF2)
 - Protection des données sensibles via le système de protection des données d'ASP.NET Core
- **Interface utilisateur intuitive** :
 - Formulaire de connexion/inscription ergonomiques
 - Gestion de profil utilisateur simplifiée
 - Workflows de récupération de mot de passe robustes

3.7.2 Autorisation basée sur les rôles

QuantumDesk implémente un modèle d'autorisation sophistiqué basé sur les rôles, permettant un contrôle précis des accès aux différentes fonctionnalités du système :

- **Hiérarchie de rôles structurée** :
 - **Admin** : Accès complet à toutes les fonctionnalités du système, incluant la gestion des utilisateurs, équipes et configurations
 - **SupportAgent** : Accès aux fonctionnalités de support (tickets, commentaires, dashboards)
 - **User** : Accès limité à ses propres tickets et à la base de connaissances
- **Contrôle d'accès déclaratif** :
 - L'attribut `[Authorize]` restreint l'accès aux utilisateurs authentifiés
 - L'attribut `[Authorize(Roles = "Admin")]` limite l'accès aux administrateurs, comme visible dans le contrôleur `Admin`
 - L'attribut `[Authorize(Roles = "Admin,SupportAgent")]` permet l'accès aux admins et agents de support
- **Autorisation programmatique** : Vérification supplémentaire dans le code pour les cas complexes, comme la vérification de propriété des tickets
- **Sécurité des vues** : Les éléments d'interface sont rendus ou masqués en fonction des rôles, avec des layouts différents selon le rôle de l'utilisateur
- **Politiques d'autorisation personnalisées** :
 - Vérification si l'utilisateur est créateur du ticket ou agent assigné
 - Vérification d'appartenance à une équipe pour l'accès aux tickets d'équipe

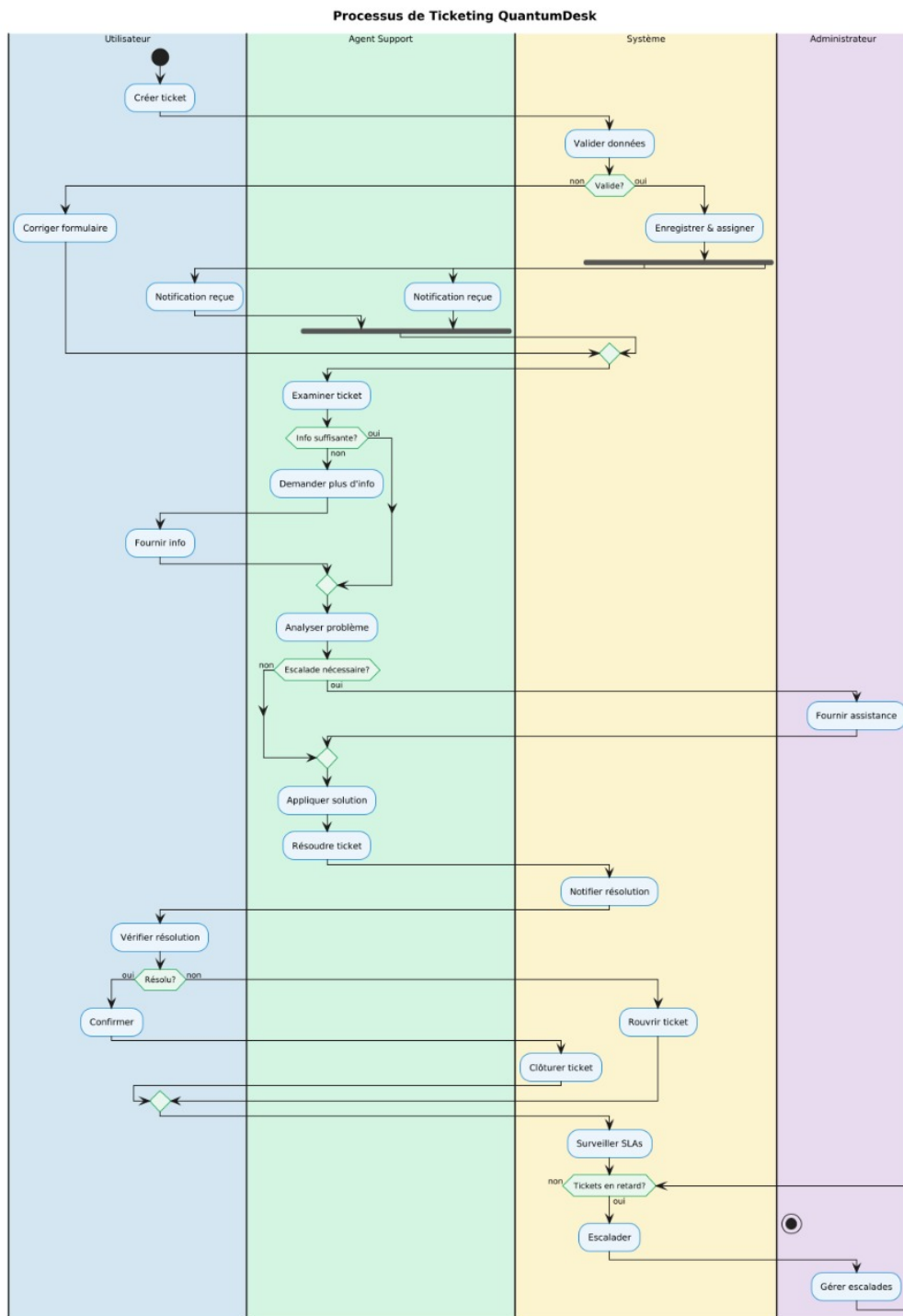


FIGURE 3.10 – Diagramme de Business Process Model and Notation de QuantumDesk

3.8 Conception de l'Interface

Introduction

L'interface de *QuantumDesk* a été conçue pour lisser la complexité fonctionnelle du *backend* et la transformer en actions rapides et compréhensibles. En s'appuyant sur la théorie de la charge cognitive de Sweller et les lois Gestalt, la hiérarchie visuelle met en avant l'information critique sans surcharger l'utilisateur. Les dix heuristiques de Nielsen guident la visibilité de l'état du système et la prévention des erreurs, tandis que le modèle de manipulation directe de Card, Moran et Newell assure un cycle action-réaction fluide. Chaque composant respecte enfin les exigences d'accessibilité WCAG 2.1 pour garantir une expérience inclusive, quel que soit le dispositif ou le profil d'utilisateur. Les sous-sections suivantes décrivent chacune des interfaces clés ; chaque description est immédiatement suivie de sa figure.

Interface Accueil (Dashboard)

L'écran d'accueil agrège les indicateurs clés – tickets ouverts, en attente, résolus – et illustre leur évolution hebdomadaire au moyen d'un graphique animé. En limitant l'affichage aux métriques essentielles, l'interface réduit la charge extrinsèque et permet à l'utilisateur de concentrer ses ressources cognitives sur l'analyse des données. Les principes de proximité et de similarité renforcent la cohérence perceptive, tandis que les éléments critiques placés dans les zones de forte attention profitent de l'effet de primauté.

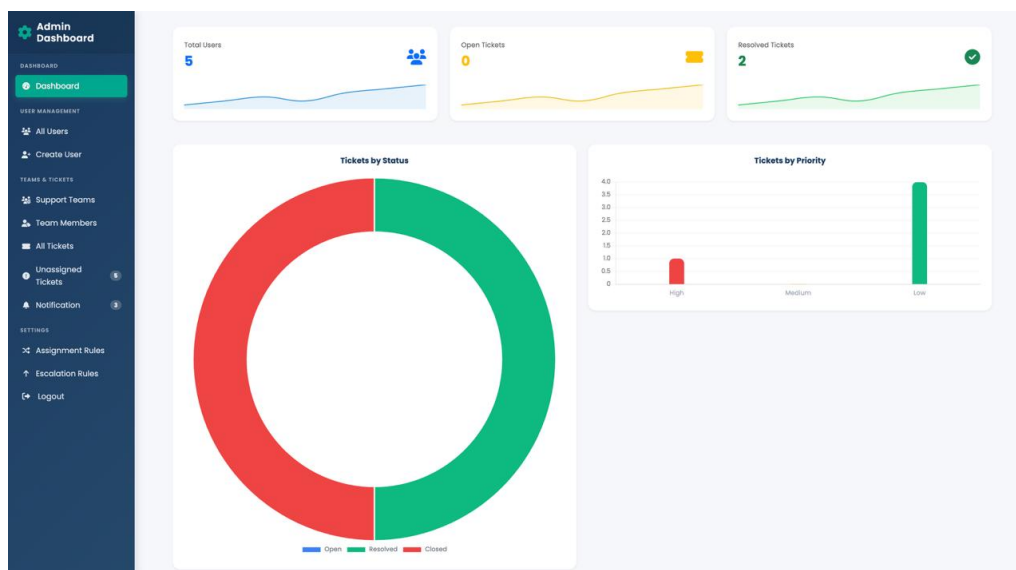


FIGURE 3.11 – Interface Accueil – Synthèse des indicateurs clés

Interface Liste des Tickets

La liste des tickets se présente sous forme tabulaire avec recherche instantanée et filtres dynamiques (statut, priorité, date). Chaque action déclenche une mise à jour immédiate, conformément au modèle de manipulation directe. Le nombre limité de choix simultanés diminue la complexité décisionnelle, selon la loi de Hick-Hyman. Les libellés explicites favorisent la reconnaissance plutôt que le rappel et réduisent la charge de mémoire de travail.

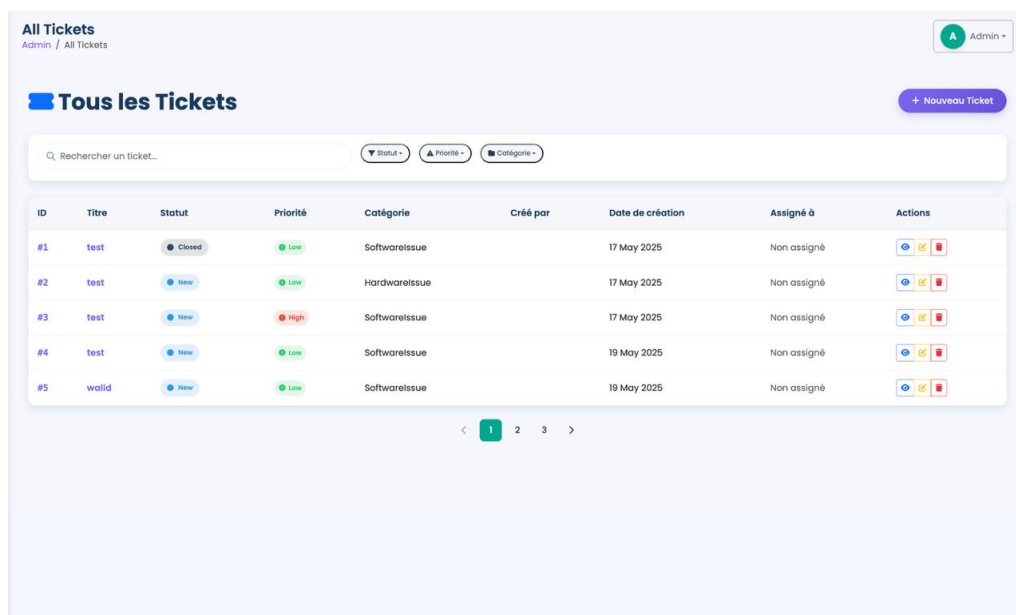


FIGURE 3.12 – Interface Liste des Tickets – Recherche et filtres dynamiques

Interface Création de Ticket

Le formulaire de création est divisé en deux blocs : informations sur la demande et données du demandeur. La validation *inline* et le feed-forward immédiat réduisent les erreurs et confèrent un sentiment de maîtrise. La dimension et la proximité des zones cliquables sont calibrées selon la loi de Fitts tandis que le *chunking* logique modère la charge intrinsèque.

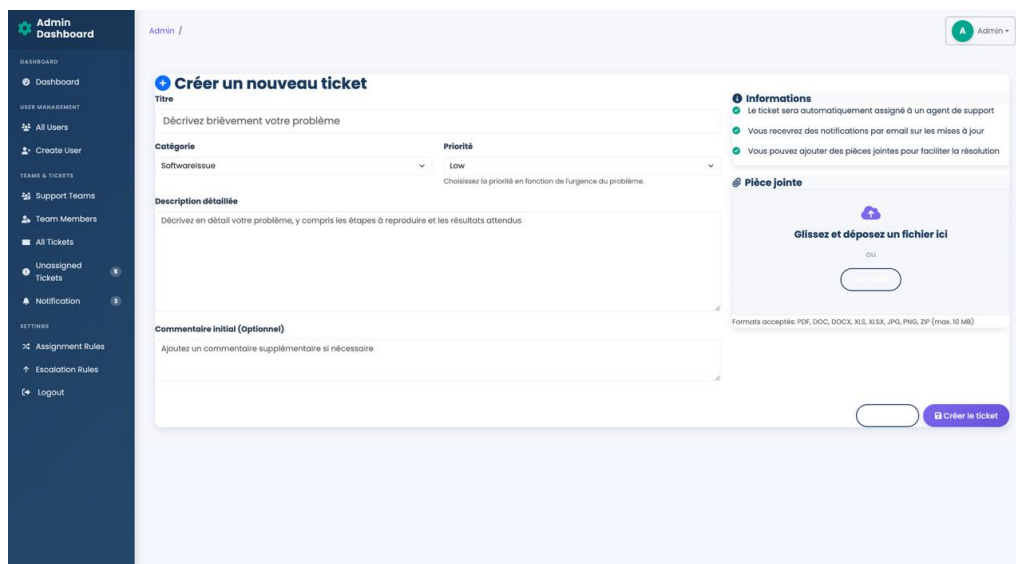


FIGURE 3.13 – Interface Création de Ticket – Formulaire guidé

Interface Détail du Ticket

La fiche détaillée organise description, commentaires, pièces jointes et historique dans des onglets modulaires. La divulgation progressive recommandée par Shneiderman évite la surcharge

d'information . Les actions contextuelles (assigner, clôturer, commenter) respectent le mapping action-résultat décrit par Norman . L'historique horodaté assure la traçabilité indispensable à la gestion d'incidents.

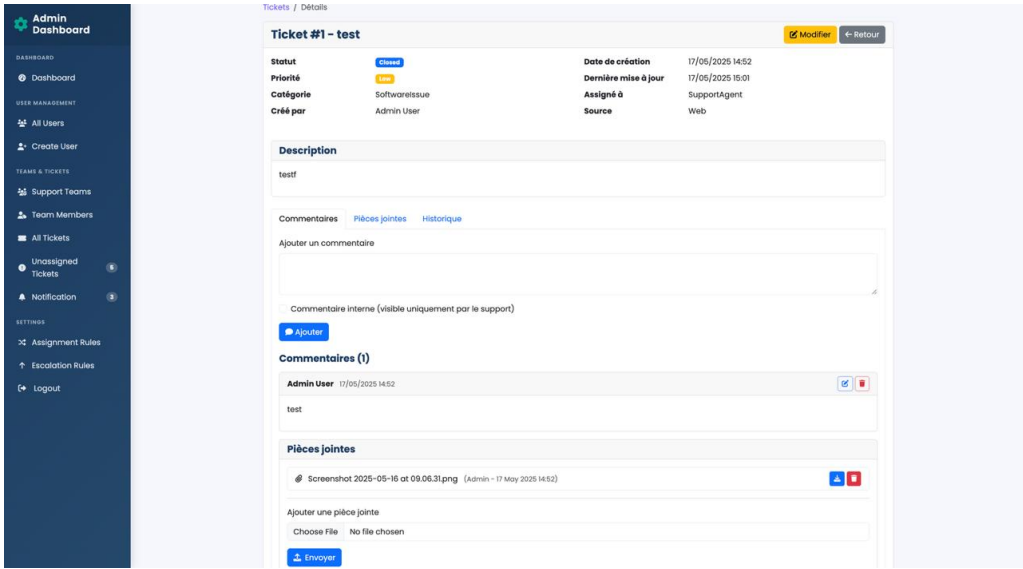


FIGURE 3.14 – Interface Détail du Ticket – Vue modulaire et historique

Interface Gestion des Équipes

Cette vue s'appuie sur un double panneau pour transférer les *SupportAgents* d'une liste à l'autre. La métaphore de transfert, cohérente avec les modèles d'affordance et de mapping, simplifie la représentation mentale de l'utilisateur . Les contraintes de rôle sont vérifiées en amont, illustrant la prévention d'erreurs prônée par Nielsen . Une recherche incrémentale répond aux exigences de performance sur de grands ensembles de données .



FIGURE 3.15 – Interface Gestion des Équipes – Attribution par double panneau

Volet Notifications et Historique

Le volet notifications, accessible à tout moment, diffuse les événements critiques en temps réel via SignalR. Il satisfait à l’heuristique de visibilité de l’état du système et alimente la mémoire externe de l’équipe grâce à un historique détaillé, concept clé de la visualisation de l’information . La séparation entre alertes instantanées et historique complet illustre le principe de divulgation progressive .

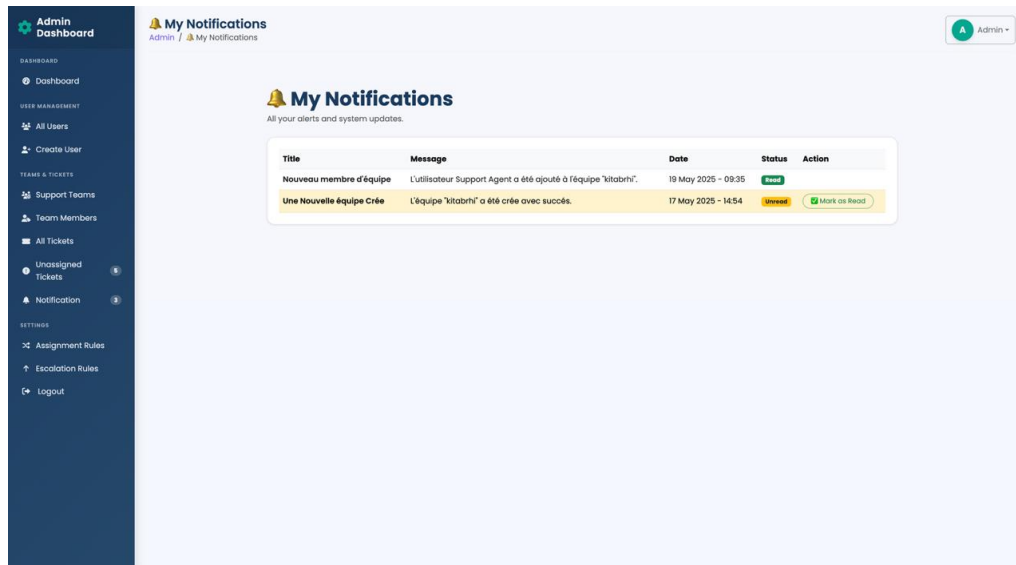


FIGURE 3.16 – Volet Notifications et Historique – Suivi temps réel

3.8.1 Expérience utilisateur responsive

L’interface a été conçue selon une approche *mobile-first*, garantissant une expérience optimale sur tous les appareils :

- **Adaptation aux différentes tailles d’écran :**
 - Bureau (>1200px) : Vue complète avec tableaux de bord et graphiques
 - Tablette (768px–1199px) : Réorganisation des blocs pour maximiser l’espace utile
 - Mobile (<767px) : Navigation simplifiée, menu hamburger, priorité aux fonctions critiques
- **Interactions tactiles optimisées :**
 - Boutons et zones d’interaction élargis
 - Alternatives aux survols pour écrans tactiles
- **Performance optimisée :**
 - Chargement différé d’images
 - Minification CSS/JS
 - Media queries intelligents

3.8.2 Vues spécialisées par rôle

L'interface utilisateur s'adapte selon le rôle de l'utilisateur connecté, affichant uniquement les fonctionnalités pertinentes :

- **Tableaux de bord administrateur :**
 - Vue globale sur tous les tickets avec filtres
 - Statistiques d'équipe et d'agents
 - Gestion des utilisateurs et des équipes
 - Paramètres système (règles d'assignation, escalation)
- **Environnement agent de support :**
 - Focus sur les tickets assignés
 - Accès aux tickets d'équipe
 - Système de notifications
 - Formulaire de réponse enrichis
- **Interface utilisateur standard :**
 - Création guidée de tickets
 - Suivi visuel de l'état des tickets
 - Accès à la base de connaissances
 - Notifications sur les mises à jour

3.9 Infrastructure DevOps et Déploiement Cloud

3.9.1 Conteneurisation avec Docker

QuantumDesk exploite pleinement les avantages de la conteneurisation Docker pour assurer une cohérence entre les environnements de développement, test et production :

- **Dockerfile optimisé :**
 - Construction multi-étapes pour réduire la taille de l'image finale
 - Séparation des étapes de build et runtime pour une sécurité accrue
 - Configuration des permissions appropriées pour les répertoires sensibles
 - Configuration des variables d'environnement pour adapter le comportement selon le contexte
 - Exposition du port HTTP 80 pour la communication
- **Docker Compose pour l'environnement complet :**
 - Orchestration de multiples services interconnectés
 - Service web ASP.NET Core avec configuration pour le hot-reload en développement
 - Instance SQL Server isolée avec persistance des données
 - Volumes mappés pour les pièces jointes et données persistantes
 - Réseau dédié pour la communication inter-services sécurisée
 - Variables d'environnement injectées pour configuration sans code
 - Healthchecks pour assurer la disponibilité des services

- **Optimisations pour la production :**
 - Images légères et optimisées
 - Gestion efficace des layers Docker pour minimiser la surface d'attaque
 - Configuration des redémarrages automatiques en cas de défaillance

3.9.2 Déploiement sur Azure App Service

La solution a été conçue spécifiquement pour un déploiement efficace sur l'infrastructure Azure :

- **Azure App Service :**
 - Déploiement sans friction des conteneurs Docker via Azure Container Registry
 - Configuration des slots de déploiement pour les stratégies blue/green deployment
 - Scaling automatique basé sur les métriques d'utilisation
 - Application des politiques de sécurité Azure au niveau applicatif
 - Intégration avec Azure Front Door pour la distribution globale
 - Configuration des domaines personnalisés et certificats SSL
- **Azure SQL Database :**
 - Migration transparente depuis la base SQL Server locale
 - Configuration des niveaux de performance adaptés à la charge
 - Backup automatisés et restauration point-in-time
 - Géo-réplication pour la haute disponibilité
 - Chiffrement transparent des données au repos
 - Audit de sécurité et détection d'anomalies
- **Stockage Azure :**
 - Utilisation des Blob Storage pour les pièces jointes volumineuses
 - CDN pour la distribution efficace des ressources statiques
 - Stratégies de rétention configurables
 - Accès sécurisé via URL signées temporaires

3.9.3 Pipeline DevOps complet

Un pipeline CI/CD complet a été implémenté pour automatiser le déploiement et garantir la qualité :

- **Azure DevOps Pipelines :**
 - Build automatisé à chaque commit sur les branches principales
 - Exécution des tests unitaires et d'intégration
 - Analyse de code statique avec SonarQube
 - Construction et publication des images Docker
 - Déploiement automatisé vers les environnements de test
 - Approval gates pour les déploiements production
 - Notification automatique des équipes en cas de problème
- **Infrastructure as Code :**

- Définition de l'infrastructure Azure en ARM Templates
- Versioning de la configuration infrastructure
- Déploiement reproductible entre environnements
- Paramétrage par environnement (Dev, Test, UAT, Production)
- **Monitoring et observabilité :**
 - Intégration avec Application Insights
 - Tableaux de bord de monitoring personnalisés
 - Alerting proactif sur les métriques critiques
 - Tracing distribué pour identifier les goulots d'étranglement
 - Analyse des performances et recommandations d'optimisation

3.9.4 Stratégie de montée en charge

L'architecture a été spécifiquement conçue pour supporter une croissance importante du trafic et des données :

- **Scalabilité horizontale et verticale :**
 - Application sans état pour faciliter la réplication horizontale
 - Support de multiples instances derrière un load balancer
 - Mise en cache distribuée avec Azure Redis Cache
 - Partitionnement de base de données pour les grands volumes
- **Stratégie de résilience :**
 - Circuit breakers pour les services externes
 - Retry policies avec exponential backoff
 - Fallback mechanisms pour les fonctionnalités critiques
 - Health checks pour auto-healing des services défaillants
- **Sécurité en profondeur :**
 - WAF (Web Application Firewall) pour protection contre les attaques OWASP Top 10
 - Scanning régulier des vulnérabilités dans les conteneurs
 - Politique de moindre privilège pour les identités service
 - Chiffrement bout-en-bout des données sensibles

Cette infrastructure DevOps complète garantit non seulement un déploiement fiable et reproductible, mais aussi une évolutivité et une maintenance facilitées sur le long terme, permettant à QuantumDesk de s'adapter efficacement à une base utilisateur croissante.

Architecture Technique et Flux de Traitement

QuantumDesk

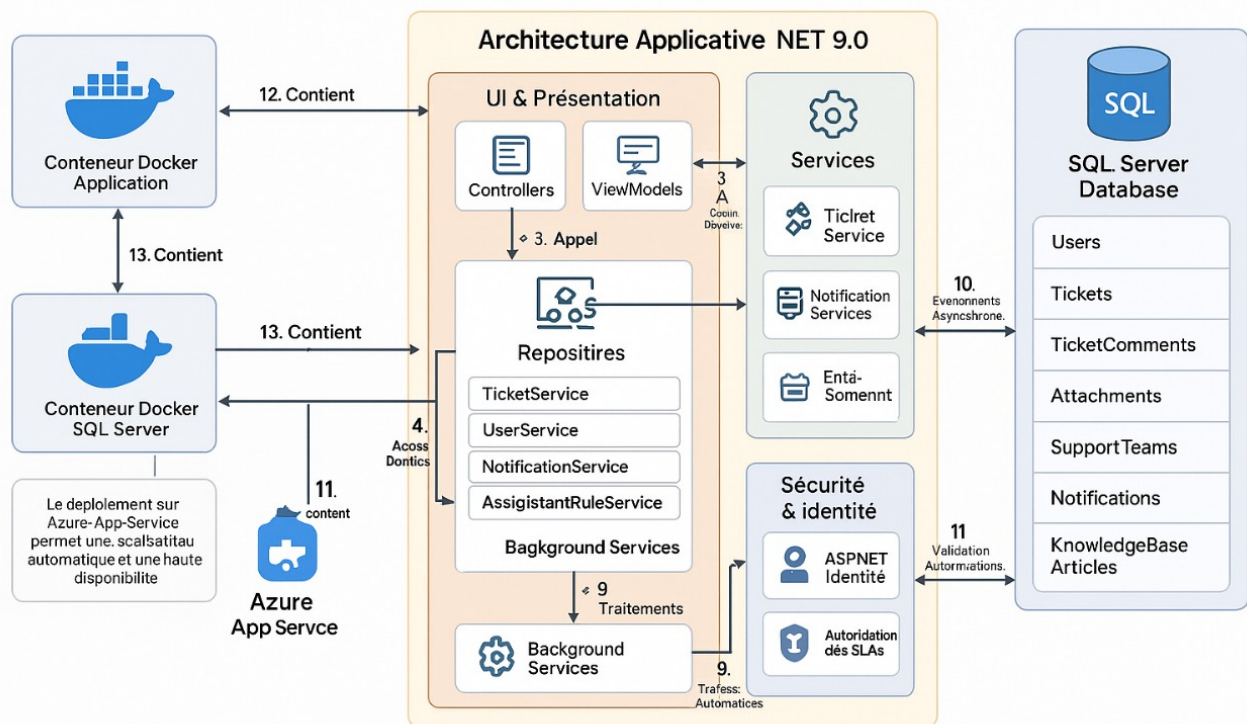


FIGURE 3.17 – Stack technologique globale utilisée pour l'application QuantumDesk

Chapitre 4

Discussion et analyse critique

4.1 Points forts de l'implémentation

Architecture modulaire et extensible

Le système *QuantumDesk* a été conçu selon une architecture en couches clairement définie, utilisant les patterns *Repository* et *Service Layer*. Cette approche offre plusieurs avantages significatifs :

- **Séparation des préoccupations** : Chaque composant a une responsabilité unique, limitant l'impact des modifications et facilitant la maintenance.
- **Abstraction de l'accès aux données** : Le pattern *Repository* isole la logique métier de la persistance.
- **Testabilité améliorée** : La modularité permet des tests unitaires ciblés avec mocks.
- **Extensibilité** : L'ajout de nouvelles fonctionnalités est facilité.

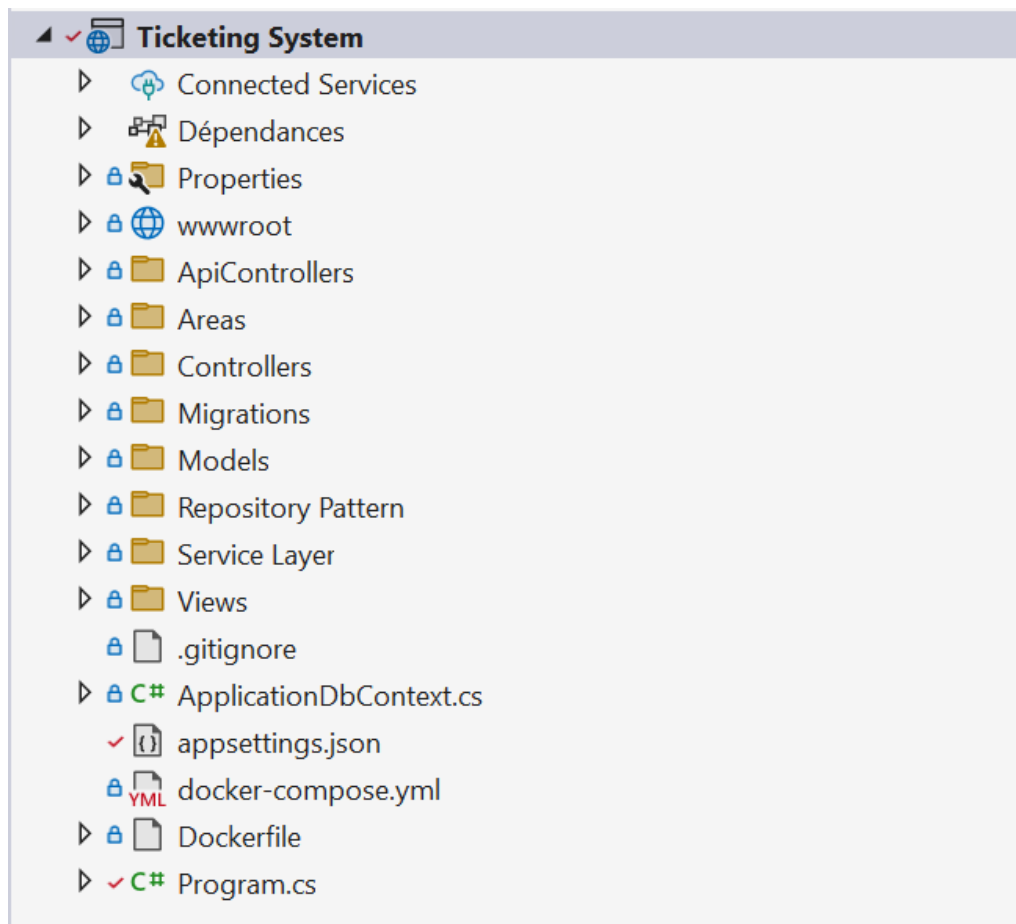


FIGURE 4.1 – Architecture en couches de QuantumDesk

Automatisation intelligente

Le système intègre des mécanismes d'automatisation avancés :

- Assignment automatique selon règles configurables
- Escalade automatique basée sur les SLA
- Ordonnancement des règles
- Services d'arrière-plan asynchrones

Interface utilisateur moderne et intuitive

- Thème *QuantumDesk* avec identité visuelle forte
- Design responsive multi-appareils
- Composants interactifs et animations
- Tableaux de bord visuels
- Accessibilité conforme WCAG

Système de notification complet

- Notifications en temps réel
- Typologie et priorité des alertes
- Centre de notifications centralisé
- Paramètres personnalisables

Documentation extensive

- Code entièrement commenté
- Structure de projet cohérente
- Documentation des API
- Guide d'administration

4.2 Défis rencontrés et solutions

Gestion des pièces jointes

Défis :

- Sécurité des fichiers
- Efficacité de stockage
- Cohérence transactionnelle
- Performance d'accès

Solutions :

- Stockage hybride base de données + système de fichiers
- Validation MIME + antivirus
- Noms de fichiers sécurisés (GUID)
- Transactions atomiques
- Vérification des permissions

Performance avec un volume croissant de tickets

Défis :

- Complexité des requêtes
- Historique très volumineux
- Chargement excessif des relations

Solutions :

- Indexation optimisée
- Lazy loading
- Pagination des listes
- Requêtes LINQ réécrites
- Mise en cache de second niveau

Complexité des règles d'assignation et d'escalade

Défis :

- Conflits entre règles
- Interface de configuration complexe
- Ordre d'évaluation des règles
- Performance du moteur de règles

Solutions :

- Attribut RuleOrder
- Interface d'administration visuelle
- Validation des conflits
- Logs détaillés
- Mode simulation



FIGURE 4.2 – Interface de test des règles d'assignation montrant le résultat simulé

4.3 Perspectives d'amélioration

Intégration avec des services externes

- **Outils de communication** : Teams, Slack, SMS, Email
- **Gestion de projet** : Jira, Azure DevOps, Trello, GitHub
- **Reporting** : PowerBI, Tableau
- **CRM / autres ticketings** : Salesforce, ServiceNow, Zendesk

Intelligence artificielle pour le triage

- **Catégorisation automatique** : Analyse sémantique, entités, mots-clés

- **Système de recommandation** : Solutions, agents, articles
- **Prédictions** : Durée, escalade, tendances
- **Chatbot** : Collecte, questions fréquentes, création assistée

API RESTful complète

- **Avantages** : App mobile, intégrations tierces, automatisation
- **Fonctionnalités clés** :
 - Authentification OAuth2
 - Documentation Swagger / OpenAPI
 - Versioning
 - Webhooks
 - Rate limiting

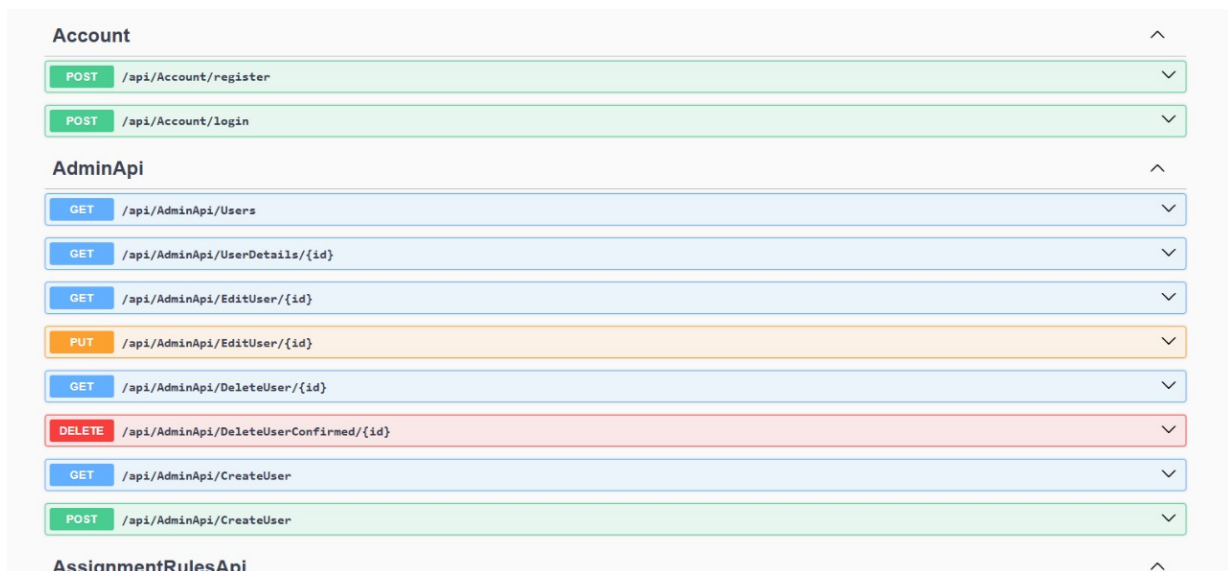


FIGURE 4.3 – Documentation Swagger/OpenAPI pour l'API RESTful de QuantumDesk

Amélioration des analyses et rapports

- Tableaux de bord interactifs avec KPIs dynamiques
- Rapports personnalisés, planifiables, exportables
- Alertes conditionnelles sur seuils dépassés
- Visualisations avancées en temps réel

Internationalisation et localisation

- Interface multilingue et traduction dynamique
- Fuseaux horaires locaux et SLA géolocalisés
- Formats régionaux (date, nombre, devise)
- Support des exigences RGPD / conformité régionale

Conclusion Générale

Le parcours des patients dans les hôpitaux est un processus complexe qui nécessite une gestion minutieuse des ressources et des flux de patients pour garantir l'efficacité, la rapidité et la qualité des soins. Ce projet a démontré l'importance de l'optimisation des flux de patients en utilisant des outils de recherche opérationnelle, notamment l'**algorithme de flot maximal**, pour améliorer la gestion des étapes critiques du parcours hospitalier.

À travers une **modélisation graphique** du parcours patient, représentant les différentes étapes comme des nœuds et les transitions entre elles comme des arcs, ce projet a permis d'identifier les **goulets d'étranglement** dans des services clés comme la radiologie et le traitement radiologique. Ces goulets ont été analysés en détail, et des solutions concrètes, telles que l'augmentation des ressources humaines et matérielles, la réorganisation des flux et l'optimisation des priorités, ont été proposées pour les résoudre.

Les résultats obtenus à partir de la simulation ont montré qu'il est possible de **maximiser le nombre de patients traités** tout en **minimisant les temps d'attente**, en tenant compte des contraintes des ressources disponibles. Le calcul du flot maximal a permis de déterminer le **nombre optimal de patients** que l'hôpital peut traiter tout en respectant les capacités des services, et a fourni des informations précieuses pour une gestion plus efficace des ressources.

Cependant, malgré les résultats prometteurs de ce modèle, certaines **limitations** ont été identifiées. La simplification du modèle par rapport à la réalité des parcours patients, ainsi que la dépendance à la qualité des données disponibles, sont des facteurs qui doivent être pris en compte lors de la mise en œuvre réelle des solutions proposées. De plus, la mise en place de certaines améliorations, comme l'augmentation des capacités des services, peut nécessiter des **investissements importants**.

Ce projet a également ouvert la voie à de nombreuses **perspectives** pour améliorer la gestion des flux hospitaliers. L'utilisation de la **modélisation dynamique** et des **technologies avancées**, telles que l'intelligence artificielle et l'analyse prédictive, pourrait permettre de mieux anticiper les besoins et d'optimiser en temps réel les parcours des patients.

En conclusion, ce projet montre que l'**optimisation des flux de patients** à l'aide de la **recherche opérationnelle** peut jouer un rôle clé dans l'amélioration de l'efficacité des hôpitaux, tout en garantissant un meilleur service aux patients et en réduisant la pression sur les professionnels de santé. Les résultats obtenus ne sont qu'une première étape, et il est essentiel de continuer à explorer et à appliquer ces méthodologies pour rendre les systèmes de santé plus résilients et adaptés aux défis futurs.

Bibliographie

- [1] Atlassian. Jira service management, 2025.
- [2] Axelos. *ITIL Foundation : ITIL 4 Edition*. The Stationery Office, 2019.
- [3] AXELOS Limited. *ITIL 4 : Foundation*. The Stationery Office, London, UK, 2nd edition, 2023.
- [4] J. Bloch. *Effective Java*. Addison-Wesley Professional, 2018.
- [5] W. Chen, M. Johnson, and S. Patel. Ai-enhanced ticket routing : improving assignment accuracy in it service management. *Journal of Service Management*, 35(2) :217–236, 2024.
- [6] Customer Satisfaction Alliance. Ticket management systems : Impact on customer satisfaction. Technical report, Customer Satisfaction Alliance, Chicago, IL, 2024.
- [7] Docker. Docker compose overview, 2023.
- [8] DoFactory. Repository design pattern, 2023.
- [9] D. Esposito and A. Saltarello. *Microsoft .NET : Architecting Applications for the Enterprise*. Microsoft Press, Redmond, WA, 3rd edition, 2023.
- [10] E. Evans. *Domain-Driven Design : Tackling Complexity in the Heart of Software*. Addison-Wesley Professional, 2003.
- [11] Forrester Research. The forrester wave : Enterprise service management, q2 2024. Technical report, Forrester Research Inc., Cambridge, MA, 2024.
- [12] M. Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, 2002.
- [13] A. Freeman. *Pro ASP.NET Core 3 : Develop Cloud-Ready Web Applications Using MVC, Blazor, and Razor Pages*. Apress, 2020.
- [14] Freshworks Inc. Freshdesk - customer service software, 2025.
- [15] HDI - Help Desk Institute. Annual support center practices & salary report. Technical report, HDI, 2024. Rapport annuel sur les pratiques des centres de support.
- [16] International Organization for Standardization. Iso/iec 20000-1 :2024 : Information technology — service management, 2024.
- [17] itSMF International. *An Introductory Overview of ITIL 2011*. The Stationery Office, 2011.
- [18] R. Martin and T. Johnson. Best practices for service level agreements in it support, 2024.
- [19] R. C. Martin. *Clean Code : A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.
- [20] R. C. Martin. *Clean Architecture : A Craftsman’s Guide to Software Structure and Design*. Prentice Hall, 2017.
- [21] Microsoft. Asp.net core identity, 2023.
- [22] Microsoft. Containerize a .net application, 2023.
- [23] Microsoft. Create web apis with asp.net core, 2023.

- [24] Microsoft. Entity framework core, 2023.
- [25] Microsoft. Security best practices for asp.net core, 2023.
- [26] O. of Government Commerce. *ITIL Service Operation*. The Stationery Office, 2018.
- [27] R. Osherove. *The Art of Unit Testing : With Examples in .NET*. Manning Publications, 2013.
- [28] OWASP. Owasp top ten, 2021.
- [29] K. Rahman, Y. Liu, and J. Wang. Smart escalation mechanisms in modern service desks. In *Proceedings of the International Conference on Service Systems*, pages 142–151, Toronto, Canada, 2023. IEEE.
- [30] ServiceNow Inc. Servicenow it service management, 2025.
- [31] J. Smith, J. Galloway, and L. Simoes. *ASP.NET Core in Action*. Manning Publications, Shelter Island, NY, 3rd edition, 2023.
- [32] D. Stuttard and M. Pinto. *The Web Application Hacker’s Handbook : Finding and Exploiting Security Flaws*. Wiley, 2011.
- [33] J. Tidwell, C. Brewer, and A. Valencia. *Designing Interfaces : Patterns for Effective Interaction Design*. O’Reilly Media, Sebastopol, CA, 4th edition, 2023.
- [34] D. Warriner and M. Hotle. Magic quadrant for it service management platforms. *Gartner Research*, August 2024.
- [35] Zendesk, Inc. Zendesk support suite, 2025.

